

Redes_e_Computadores (1)

Capítulo 1: Redes de Computadores e a Internet

1.1 O que é a Internet?

1.1.1 Descrição por Componentes (*Nuts-and-Bolts*)

A Internet é a maior infraestrutura de engenharia construída pela humanidade. Ela é composta por três elementos de *hardware* e *software* fundamentais:

1. **Sistemas Finais (*Hosts* ou *End Systems*):** Dispositivos conectados à rede que executam aplicações na periferia (borda). Incluem computadores, servidores em *datacenters*, *smartphones*, veículos conectados e dispositivos IoT.
2. **Enlaces de Comunicação (*Links*):** Meios físicos (fibra óptica, cabo coaxial, par trançado, rádio) que transportam dados entre dispositivos a uma determinada **taxa de transmissão** ou largura de banda (R), medida em **bits por segundo (bits/s)**.
3. **Comutadores de Pacotes (*Packet Switches*):** Equipamentos intermediários no núcleo da rede que recebem blocos de dados (**pacotes**), examinam seus cabeçalhos e os encaminham em direção ao destino. Os dois tipos principais são os **roteadores** (camada de rede) e os **switches de enlace** (camada de enlace).

Os sistemas finais conectam-se à Internet por meio de **Provedores de Serviços de Internet (ISPs)** de diferentes níveis (locais, regionais e globais).

1.1.2 Descrição por Serviços (*Services Description*)

Sob a perspectiva das aplicações, a Internet é uma **infraestrutura que fornece serviços** para aplicações distribuídas (Web, *streaming* de vídeo, jogos *online*, e-mail, e-commerce).

Ela disponibiliza para os programas finais uma **Interface de Programação de Aplicação (API)** — um conjunto de regras que permite que um programa rodando em um sistema final solicite à infraestrutura da Internet a entrega de dados a outro programa rodando em outro hospedeiro.

1.1.3 O que é um Protocolo?

Toda a atividade de comunicação na Internet é governada por **protocolos**.

- **Analogia Humana:** Quando você deseja saber as horas, diz "Com licença, que horas são?". Se a outra pessoa entender o idioma e a convenção social, responderá com a hora. Se você falar em um idioma desconhecido, a comunicação falha.

- **Definição de Rede:** Um **protocolo** define o **formato** e a **ordem** das mensagens trocadas entre duas ou mais entidades comunicantes na rede, bem como as **ações tomadas** na transmissão e/ou recepção de uma mensagem ou outro evento.
-

1.2 A Borda da Rede (*Network Edge*)

A borda da rede é o local onde ficam os sistemas finais (*hosts*), que abrigam os programas de aplicação (clientes e servidores).

```
[ Aplicações / Hosts ]<--->[ Rede de Acesso ]<--->[ Núcleo da Rede (Roteadores) ]  
      (Borda da Rede)           (DSL, FTTH, Wi-Fi)           (Comutação de Pacotes)
```

1.2.1 Redes de Acesso (*Access Networks*)

A rede de acesso é a infraestrutura física que conecta fisicamente um sistema final ao seu primeiro roteador (*edge router*) no caminho para o núcleo. As principais tecnologias são:

- **DSL (*Digital Subscriber Line*):** Utiliza a linha telefônica de par trançado de cobre existente para transmitir dados em frequências mais altas do que a voz.
 - **Cabo / HFC (*Hybrid Fiber-Coaxial*):** Utiliza a infraestrutura de TV a cabo com multiplexação por divisão de frequência (FDM). Múltiplos canais de dados e TV compartilham o mesmo cabo coaxial até um nó óptico residencial.
 - **FTTH (*Fiber to the Home*):** Leva a fibra óptica diretamente da central telefônica até a residência, oferecendo taxas de transmissão de centenas de Mbps a Gbps por meio de arquiteturas como PON (*Passive Optical Network*).
 - **Wireless e Celular:**
 - **Wi-Fi (802.11):** Acesso sem fio de curto alcance conectado a um Ponto de Acesso (AP) que se conecta à rede cabeada.
 - **4G LTE / 5G:** Acesso sem fio de grande área conectado a torres de celular (*base stations*) operadas por provedores móveis.
-

1.2.2 Meios Físicos de Transmissão (*Physical Media*)

A informação física navega pela rede na forma de ondas eletromagnéticas ou impulsos ópticos guiados ou não guiados:

1. **Meios Guiados (*Guided Media*):** O sinal se propaga contido dentro de um meio sólido.
 - **Par Trançado de Cobre (*Twisted Pair - UTP*):** O meio mais comum para redes locais (Ethernet). Os fios são trançados em pares para reduzir a interferência eletromagnética de

pares vizinhos.

- **Cabo Coaxial:** Formado por dois condutores de cobre concêntricos. Suporta taxas mais altas e transmissão banda larga (múltiplos canais por FDM).
- **Fibra Óptica:** Conductor de vidro flexível e fino que carrega pulsos de luz. Suporta taxas extremamente altas (dezenas a centenas de Gbps) com imunidade a interferências eletromagnéticas e atenuação muito baixa por quilômetro.

2. **Meios Não Guiados (*Unguided Media*):** O sinal se propaga livremente na atmosfera ou no espaço.

- **Rádio:** Onda eletromagnética enviada pelo ar. Sofre com degradação devido a barreiras físicas, reflexão e interferência de outros dispositivos. Exemplos: canais de rádio de redes Wi-Fi, celulares e enlaces por satélite (LEO e GEO).

1.3 O Núcleo da Rede (*Network Core*)

O núcleo da rede é a malha interconectada de comutadores de pacotes (*roteadores* e *switches*) e enlaces que conectam as redes de acesso entre si. Sua função primária é transportar dados de um sistema final de origem para um de destino através de nós intermediários.

1.3.1 Comutação de Pacotes (*Packet Switching*)

Conceito

Na comutação de pacotes, os sistemas finais dividem as mensagens em blocos discretos de dados chamados **pacotes**. Cada pacote atravessa a rede transitando por enlaces e comutadores de pacotes. Os dados são transmitidos à taxa total do enlace de saída.

1. Transmissão Armazena-e-Reenvia (*Store-and-Forward*)

A maioria dos comutadores de pacotes opera segundo o princípio de **armazena-e-reenvia**: o comutador precisa receber o pacote por inteiro em seu *buffer* de entrada antes de poder começar a transmitir o primeiro bit do pacote para o enlace de saída.

- **Análise de Atraso Simples:** Se enviarmos 1 pacote de (L) bits através de (N) enlaces idênticos em série, cada um com taxa de transmissão (R) (e ignorando a propagação por enquanto), o atraso fim a fim é:

2. Atraso de Fila e Perda de Pacotes

Cada comutador possui múltiplos enlaces de saída. Para cada enlace, o comutador mantém um **buffer de saída** (ou fila):

- **Formação de Fila:** Se um pacote chega e encontra o enlace ocupado transmitindo outro pacote, ele precisa aguardar no *buffer*.
- **Perda de Pacote (*Packet Loss*):** Como a memória do *buffer* é finita, se um pacote chegar e encontrar o *buffer* completamente cheio, o comutador não terá onde armazená-lo e o descartará (*drop*).

3. Tabelas de Repasse e Protocolos de Roteamento

Como o roteador sabe para qual enlace de saída enviar determinado pacote?

- Cada pacote contém em seu cabeçalho o **endereço IP de destino**.
 - O roteador possui uma **tabela de repasse** (*forwarding table*) que mapeia endereços IP de destino para seus enlaces de saída físicos.
 - As tabelas de repasse são construídas automaticamente por **protocolos de roteamento** (como OSPF e BGP) que calculam os caminhos mais curtos e eficientes através da malha.
-

1.3.2 Comutação de Circuitos (*Circuit Switching*)

Conceito

Na comutação de circuitos, os recursos necessários ao longo de um caminho para prover a comunicação entre os sistemas finais (largura de banda, tempos de comutação) são **reservados e dedicados exclusivamente** durante toda a duração da sessão de comunicação.

```
[ Usuário A ] ===(Recursos Reservados/Dedicados)=== [ Usuário B ]
```

Multiplexação na Comutação de Circuitos

Para permitir que múltiplos circuitos compartilhem o mesmo cabo físico, utilizam-se duas técnicas clássicas de multiplexação:

1. **FDM (*Frequency-Division Multiplexing*):** O espectro de frequência do meio é dividido em faixas fixas. Cada conexão recebe sua própria faixa dedicada.
 2. **TDM (*Time-Division Multiplexing*):** O tempo é dividido em quadros (*frames*) com número fixo de compartimentos de tempo (*slots*). Cada conexão recebe um *slot* exclusivo por quadro.
-

1.3.3 Comparação e Unificação Matemática: Multiplexação Estatística (Kurose + UFRJ)

Fenômeno de Rede

Por que a Internet moderna utiliza comutação de pacotes em vez de comutação de circuitos? Os usuários de rede não transmitem dados continuamente; o tráfego de aplicações Web, e-mail e áudio/vídeo é essencialmente **em rajadas** (*bursty*) — alternando períodos curtos de transmissão intensa com longos períodos de ociosidade.

Unificação Matemática 1: O Modelo Binomial de Usuários Ativos

1. O que representa?

Modela o número aleatório de usuários ativamente transmitindo dados ao mesmo tempo em um enlace compartilhado por uma população finita de usuários independentes.

2. Por que estamos usando esse modelo?

Para provar quantitativamente por que a comutação de pacotes permite admitir **muito mais usuários** na mesma infraestrutura do que a comutação de circuitos, mantendo a probabilidade de congestionamento (*sobrecarga*) extremamente baixa.

3. Significado das Variáveis e Unidades

- m (ou N): Número total de usuários independentes que compartilham o enlace.
- p : Probabilidade de um usuário individual estar ativo (transmitindo dados) em um dado instante.
- A (ou X): Variável aleatória discreta que representa o **número de usuários ativos simultaneamente** em um dado instante.
- R : Capacidade total do enlace de saída (ex:).
- r (ou R): Número máximo de usuários simultâneos que a rede consegue atender sem formar fila (ex: se cada usuário precisa de , um enlace de suporta usuários dedicados).

4. Hipóteses do Modelo

- Os m usuários comportam-se de maneira totalmente **independente**.
- Cada usuário tem a mesma probabilidade p de estar ativo em qualquer instante.

5. Aplicação da Fórmula

Como cada usuário está ativo (1) com probabilidade p ou inativo (0) com probabilidade $1-p$, o número total de ativos A segue uma **Distribuição Binomial**:

A probabilidade de haver exatamente a usuários ativos no mesmo instante é dada pela Função de Massa de Probabilidade (PMF):

A **Carga Esperada** (número médio de usuários ativos) é:

- **Em Comutação de Circuitos:** Admitimos no máximo $m = r$ usuários. Se $r=10$, apenas 10 usuários podem ser aceitos, mesmo que passem a maior parte do tempo em silêncio.

- **Em Comutação de Pacotes:** Permitimos que $m > r$ usuários compartilhem o enlace. Haverá **sobrecarga/congestionamento** quando $A > r$. A probabilidade de sobrecarga $P(A > r)$ é a cauda da distribuição binomial:

6. Cálculo e Interpretação (Exemplo Clássico Kurose / Prova UFRJ)

Considere um enlace de , onde cada usuário consome quando ativo, mas fica ativo apenas durante 10%) do tempo ().

1. **Comutação de Circuitos:** Cada usuário requer reservados. O enlace suporta estritamente $r = 10$ usuários. O 11º usuário é bloqueado.
 2. **Comutação de Pacotes:** Conectamos **$m = 35$ usuários** no mesmo enlace!
 - Carga média esperada: $E[A] =$ usuários ativos em média.
 - Probabilidade de sobrecarga (mais de 10 usuários ativos simultaneamente): =
- **Interpretação:** Em do tempo, há 10 ou menos usuários ativos e a rede funciona sem nenhum atraso de fila. Suportamos **3,5 vezes mais usuários** do que na comutação de circuitos com uma taxa de sobrecarga insignificante (menos de 4 vezes em 10.000 instantes). Isso é o poder da **multiplexação estatística**.
-

Unificação Matemática 2: A Aproximação de Poisson (UFRJ)

1. O que representa?

Modela o número de usuários ativos quando a população total de usuários (m) é muito grande, mas a probabilidade de cada um estar ativo (p) é muito pequena.

2. Por que estamos usando esse modelo?

Calcular somatórios binomiais com números combinatórios para ou mais é computacionalmente custoso e suscetível a estouros de precisão. A distribuição de Poisson fornece uma aproximação analítica rápida e precisa.

3. Significado das Variáveis e Unidades

- : Carga média total mantida constante (unidade: número médio de usuários ativos).

4. Hipóteses

- (grande número de fontes).
- (cada fonte raramente ativa).
- permanece moderado.

5. Aplicação da Fórmula

Substituindo o limite da Binomial quando $n \rightarrow \infty$, obtemos a PMF da **Poisson**:

Unificação Matemática 3: O Tempo de Descongestionamento — Modelo Geométrico (Notas UFRJ)

1. O que representa?

Modela o número de *slots* de tempo necessários (S) para que a rede saia do estado de sobrecarga e volte a ficar descongestionada.

2. Por que estamos usando esse modelo?

A variável binomial A tira apenas uma "fotografia" estática de um instante da rede. A variável temporal S responde à pergunta do operador de rede: *"Dado que a rede congestionou no slot atual, quantos slots levaremos, em média, até a rede voltar ao normal?"*

3. Significado das Variáveis e Unidades

- q : Probabilidade de um *slot* estar descongestionado (sucesso, ou seja, 1).
- $1 - q$: Probabilidade de o *slot* estar congestionado (0).
- S : Número de *slots* futuros até observar o primeiro *slot* descongestionado (1).

4. Hipóteses

- Os estados da rede em *slots* consecutivos são temporalmente **independentes**.

5. Aplicação das Fórmulas

Como estamos contando o número de tentativas até obter o primeiro sucesso (slot descongestionado), S segue uma **Distribuição Geométrica**:

O **Tempo Médio de Recuperação (Esperança de S)** é:

6. Interpretação em Termos de Redes

Se a probabilidade de um slot estar normal for q , o tempo médio de descongestionamento é $\frac{1}{q}$. Se a rede estiver extremamente sobrecarregada com $q \rightarrow 0$, levará em média $\frac{1}{q}$ para se recuperar.

Packet Pair

Para estimar a capacidade física do enlace mais lento (o **gargalo**) ao longo de um caminho sem depender de acesso direto aos roteadores intermediários, envia-se um par de pacotes de mesmo tamanho em sequência imediata.

1. O Fenômeno:

- Na emissão, os pacotes são enviados com um espaçamento inicial de entrada .

2. A Matemática do Espaçamento de Saída:

- A dispersão temporal observada no receptor atende à relação:

3. Análise dos Casos:

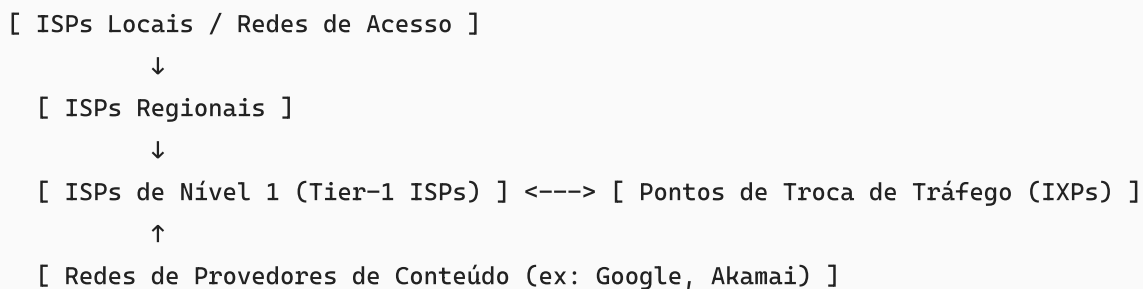
- **Enlace 2 é o Gargalo ()**: O primeiro enlace injeta o segundo pacote no roteador antes que o primeiro pacote termine de ser transmitido no segundo enlace. O segundo pacote fica retido no *buffer* por um tempo de fila , e os pacotes saem separados por .
- **Primeiro Enlace é o Gargalo ()**: O segundo enlace transmite o primeiro pacote mais rápido do que o tempo que o primeiro enlace leva para entregar o segundo pacote. Não há formação de fila (o roteador fica temporariamente ocioso), e a dispersão original é mantida: .

4. Estimativa de Capacidade:

- O receptor calcula a estimativa da capacidade do gargalo medindo o intervalo :

1.3.4 Redes de Redes (A Network of Networks)

Como os bilhões de sistemas finais se conectam globalmente? A Internet não é mantida por um único provedor, mas é uma **estrutura hierárquica de ISPs interconectados**:



1. **ISPs de Nível 1 (Tier-1 ISPs)**: Provedores globais com cobertura internacional (ex: AT&T, Cogent, NTT). Eles se interconectam diretamente por meio de enlaces privados de **peering** (sem custo de tráfego mútuo).
2. **Pontos de Troca de Tráfego (IXPs - Internet Exchange Points)**: Estruturas físicas onde múltiplos ISPs de nível 1, 2 e redes de conteúdo se reúnem para trocar tráfego diretamente, reduzindo custos de trânsito.
3. **Redes de Provedores de Conteúdo (Content Provider Networks)**: Grandes empresas (como Google, Microsoft e Facebook) constroem suas próprias redes privadas de fibra óptica e *datacenters*, ignorando os ISPs de nível superior e conectando-se diretamente aos ISPs locais/regionais para entregar conteúdo com menor latência.

1.4 Atraso, Perda e Vazão em Redes de Comutação de Pacotes

1.4.1 Visão Geral dos Atrasos Nodal e Fim a Fim

Fenômeno de Rede

Quando um pacote transita da origem ao destino, ele passa por uma sequência de roteadores intermediários. Em cada nó, o pacote não é repassado instantaneamente; ele sofre uma série de atrasos físicos e operacionais.

Conceito

O **Atraso Nodal Total** () em um único comutador de pacotes é a soma de quatro componentes fundamentais:

```
[ Chegada ] ----> [ Processamento (d_proc) ] ----> [ Fila de Espera (d_fila) ] ----> [ Transmissão (d_trans) ] ----> [ Enlace (d_prop) ] ----> [ Próximo Nó ]
```

1. **Atraso de Processamento ()**: Tempo que a CPU do roteador leva para examinar o cabeçalho do pacote, verificar erros de *checksum* e determinar a interface de saída pela tabela de repasse. Tipicamente na ordem de **microsegundos** ou menos.
2. **Atraso de Fila (ou)**: Tempo que o pacote passa aguardando no *buffer* de saída enquanto outros pacotes à sua frente são transmitidos. Varia dinamicamente de até milissegundos dependendo do tráfego.
3. **Atraso de Transmissão (ou)**: Tempo para injetar (serializar) todos os bits do pacote no enlace. .
4. **Atraso de Propagação ()**: Tempo para um único bit percorrer a distância física do meio à velocidade da luz no meio. .

1.4.2 Atraso de Fila e Intensidade de Tráfego

Fenômeno de Rede

Por que a fila se forma? Se a taxa na qual os bits chegam à porta de saída do roteador for maior do que a taxa na qual o enlace consegue transmiti-los, os pacotes precisam ser armazenados temporariamente no *buffer*.

Conceito

A métrica fundamental para avaliar o estado de congestionamento de um nó é a **Intensidade de Tráfego** (, também denotada por ρ em teoria de filas).

Unificação Matemática 1: Intensidade de Tráfego e Condição de Estabilidade

1. O que representa?

A razão entre a taxa média de chegada de bits e a capacidade de transmissão do enlace. Representa também a **utilização do enlace** (), ou seja, a fração de tempo em que o enlace fica ocupado transmitindo dados.

2. Por que estamos usando este modelo?

Para estabelecer o limite físico sob o qual uma rede opera de forma estável.

3. Significado das Variáveis e Unidades

- λ (ou λ_s): Taxa média de chegada de pacotes ao nó (λ).
- L : Tamanho médio dos pacotes (L).
- μ : Taxa de transmissão do enlace (μ).
- ρ : Taxa média de serviço do enlace (ρ).
- τ : Tempo médio de serviço/transmissão de um pacote (τ).

4. Hipóteses

- O sistema opera em **equilíbrio estacionário** (o que entra sai).
- O *buffer* possui capacidade ilimitada para armazenar pacotes.

5. Aplicação da Fórmula

- **Se** $\rho \geq 1$: Chega mais trabalho do que o enlace pode processar. A fila cresce sem limite e o atraso tende ao infinito (∞).
 - **Se** $\rho < 1$: O sistema é estável. A utilização do servidor é ρ .
 - **Se** $\rho < 1$: O sistema só é estável se as chegadas e os tempos de serviço forem estritamente **determinísticos** ($\delta = 0$) sem nenhuma variabilidade. Na presença de aleatoriedade, causa estouro de fila.
-

Unificação Matemática 2: A Lei de Little (UFRJ)

1. O que representa?

Uma lei fundamental e universal da teoria de filas que relaciona a quantidade média de entidades no sistema com a taxa de entrada e o tempo médio de permanência.

2. Por que estamos usando este modelo?

Porque ela **não depende** da distribuição probabilística do tráfego nem da regra de atendimento; ela se aplica a qualquer sistema em equilíbrio.

3. Significado das Variáveis e Unidades

- λ (ou λ_s): Taxa média efetiva de chegada de pacotes (λ).
- L : Tempo total médio que um pacote passa no sistema (fila + serviço) (L).
- τ (ou τ_s): Tempo médio de espera do pacote estritamente na **fila** (τ).
- τ_s : Tempo médio de transmissão no **servidor** (τ_s).

- : Número médio de pacotes no **sistema completo** (L).
- (ou L_q): Número médio de pacotes aguardando na **fila**.
- : Número médio de pacotes em transmissão no **servidor** (L_s).

4. Hipóteses

- O sistema está em equilíbrio estatístico e conservação de taxa ($\rho < 1$).

5. Aplicação das Fórmulas de Little

1. **Ao Sistema Completo:**
2. **Apenas à Fila de Espera:**
3. **Apenas ao Servidor:**

Unificação Matemática 3: O Modelo de Fila (Gabarito Provas UFRJ)

1. O que representa?

Modela o atraso e a ocupação de um buffer de roteador assumindo **chegadas de Poisson** (λ) e **tempos de transmissão exponenciais** (μ , ou seja, tamanhos de pacote aleatórios).

2. Por que estamos usando este modelo?

Fornece uma expressão analítica exata para prever como o atraso de fila cresce de forma assintótica conforme a utilização se aproxima de 1.

3. Hipóteses do Modelo

- **Chegadas de Poisson** (propriedade **PASTA**: *Poisson Arrivals See Time Averages* — o pacote que chega enxerga a média temporal do estado da rede).
- **Tempos de serviço exponenciais** (propriedade de **falta de memória** — o tempo residual de transmissão do pacote atual é em média igual ao tempo total de serviço $1/\mu$).

4. Derivação Passo a Passo (Exigida nas Provas da UFRJ)

Um pacote recém-chegado encontra, em média, L_q pacotes na fila e L_s pacotes no servidor. Pela falta de memória e pela propriedade PASTA, seu tempo médio de espera na fila é:

Substituindo a Lei de Little :

Isolando :

Aplicando as demais métricas pela Lei de Little:

- **Tempo Total no Sistema (L):**
- **Número Médio de Pacotes no Sistema (L):**

- Número Médio de Pacotes na Fila ($\bar{n}$):
-

Unificação Matemática 4: O Modelo de Fila (Pacotes Fixos)

1. O que representa?

Modela a fila em um roteador onde todos os pacotes têm **tamanho fixo/determinístico** de bits (serviço).

2. Por que estamos usando este modelo?

Para demonstrar o **efeito da variabilidade do tamanho dos pacotes** no atraso da rede.

3. Hipóteses e Aplicação da Fórmula

Como todos os pacotes têm exatamente o mesmo tamanho, o tempo residual de transmissão que um pacote recém-chegado encontra no servidor é, em média, **metade do tempo de transmissão** ($\frac{1}{2}T$).

Repetindo a dedução da UFRJ com tempo residual :

6. Interpretação em Termos de Redes

Padronizar o tamanho dos pacotes na rede (tornando-os determinísticos) **reduz o tempo de espera na fila exatamente pela metade** em relação a uma rede com tamanhos de pacotes altamente variáveis (∞).

1.4.3 Perda de Pacotes (*Packet Loss*)

Fenômeno de Rede

Na prática, a memória do *buffer* de um roteador é **finita**. O que acontece quando chegam pacotes e a fila já está 100% cheia?

Conceito

Como não há espaço físico para armazenar o pacote recebido, o roteador simplesmente o **descarta** (*packet drop*).

- **Do ponto de vista do sistema final:** O pacote foi transmitido, mas "sumiu" no núcleo da rede.
 - A fração de pacotes perdidos aumenta exponencialmente conforme a intensidade de tráfego se aproxima ou excede .
 - Protocolos da camada de transporte confiável (como o TCP) precisam detectar essa perda e retransmitir o pacote, gerando atraso adicional para a aplicação.
-

1.4.4 Atraso Fim a Fim (*End-to-End Delay*)

Acumulando os atrasos nodais ao longo de um caminho com roteadores intermediários (enlaces idênticos) sem congestionamento:

Ferramenta Prática de Diagnóstico: Traceroute

O programa **Traceroute** permite medir o atraso de ida e volta (RTT) real até cada roteador ao longo do caminho enviando múltiplos pacotes com valores crescentes de TTL (*Time-To-Live*). Se um roteador não responder ou descartar pacotes devido a congestionamento, o Traceroute exibe um asterisco (*).

1.4.5 Vazão em Redes de Computadores (*Throughput*)

Fenômeno de Rede

A qual taxa efetiva (em bits por segundo) um arquivo é transferido da origem ao destino através de uma sequência de enlaces compartilhados?

Conceito e O Enlace Gargalo (*Bottleneck Link*)

- **Vazão Instantânea:** A taxa em um dado instante de tempo.
- **Vazão Média:** Se a transferência de um arquivo de bits leva segundos, a vazão média é .

Modelo Matemático: A Vazão em Enlaces em Série e Compartilhados

1. Aplicação da Fórmula

Considere um caminho com enlaces em série com taxas de transmissão . A vazão fim a fim $\tilde{\alpha}$ é dada estritamente pela taxa do **enlace mais lento** do caminho (o **gargalo**):

$\tilde{\alpha}$

```
[ Servidor ] ---( R_1 = 100 Mbps )---> [ Roteador ] ---( R_2 = 1.5 Mbps )----> [
Roteador ] ---( R_3 = 10 Mbps )----> [ Cliente ]
```

↑

ENLACE GARGALO (Vazão = 1.5 Mbps)

2. Efeito de Enlaces Compartilhados e Caching (Notas UFRJ)

Se o enlace gargalo de taxa for compartilhado por conexões simultâneas, a vazão de cada conexão é .

Ao instalar um **servidor Cache (Web Cache)** próximo aos clientes com uma taxa de acerto (*hit rate*) , reduzimos a taxa de chegada no enlace de acesso de para uma taxa efetiva :

Isso reduz drasticamente a intensidade de tráfego , retirando o enlace da zona de congestionamento e reduzindo o atraso de fila de segundos para milissegundos!

1.5 Camadas de Protocolos e Seus Modelos de Serviço

1.5.1 Arquitetura em Camadas (Pilhas de Protocolos)

Fenômeno de Rede

A Internet é um sistema composto por bilhões de dispositivos, aplicações heterogêneas, comutadores e enlaces físicos de tecnologias distintas. Como projetar e manter uma infraestrutura tão vasta sem que uma alteração em uma placa de rede ou navegador quebre todo o sistema?

Conceito

Para estruturar o projeto, os projetistas organizam os protocolos (e o *hardware/software* que os executam) em **camadas**.

- **Abstração de Serviços:** Cada camada oferece um **modelo de serviço** bem definido para a camada imediatamente superior.
- **Encapsulamento de Funções:** Uma camada realiza suas tarefas executando ações internas e utilizando os serviços fornecidos pela camada inferior.
- **Vantagem:** Modularidade. Se o protocolo de uma camada for atualizado (por exemplo, mudando de Wi-Fi para Ethernet na camada de enlace), as camadas superiores (como a camada de aplicação) continuam funcionando sem necessidade de modificação.
- **Desvantagem / Trade-off:** Uma camada pode duplicar funcionalidades de camadas inferiores (por exemplo, checagem de erros no enlace e no transporte) ou precisar de informações contidas em outra camada.

A Pilha de Protocolos da Internet (5 Camadas)

A arquitetura da Internet é organizada em **5 camadas**, abordada neste curso de cima para baixo (*top-down*):

+-----+		
5. Aplicação (Mensagem)	<--->	HTTP, SMTP, DNS
+-----+		
4. Transporte (Segmento)	<--->	TCP, UDP
+-----+		
3. Rede (Datagrama)	<--->	IP, Protocolos de Roteamento
+-----+		
2. Enlace (Quadro / Frame)	<--->	Ethernet, Wi-Fi, PPP
+-----+		

1. Física (Bits)	<---> Sinais elétricos / ópticos / rádio
+-----+	

1. **Camada de Aplicação:** Onde residem as aplicações de rede e seus protocolos (HTTP para Web, SMTP para e-mail, DNS para tradução de nomes). A unidade de dados é a **mensagem**.
2. **Camada de Transporte:** Transporta as mensagens da aplicação entre os processos dos sistemas finais. Na Internet, destaca-se o **TCP** (serviço confiável, orientado à conexão e com controle de congestionamento) e o **UDP** (serviço sem conexão, não confiável e sem controle de fluxo). A unidade de dados é o **segmento**.
3. **Camada de Rede:** Responsável por mover pacotes chamados **datagramas** de um *host* de origem a um de destino através dos roteadores no núcleo da rede. Contém o **Protocolo IP** e as tabelas de roteamento.
4. **Camada de Enlace:** Transfere um datagrama de um nó (host ou roteador) para o nó seguinte ao longo do caminho. Exemplo: Ethernet, Wi-Fi. A unidade de dados é o **quadro** (*frame*).
5. **Camada Física:** Move os bits individuais contidos no quadro ao longo do meio físico de transmissão (fibra, cabo trançado, ar).

O Modelo de Referência OSI (7 Camadas) vs. Pilha TCP/IP

O modelo da ISO/OSI possui **7 camadas**, adicionando duas camadas entre a Aplicação e o Transporte:

- **Camada de Apresentação:** Provê serviços como formatação de dados, compressão e criptografia.
- **Camada de Sessão:** Provê delimitação e sincronização do fluxo de dados (marcação de *checkpoints*).

Na Internet (TCP/IP), essas funções não possuem camadas dedicadas. Se uma aplicação precisar de compressão, criptografia ou controle de sessão, o próprio desenvolvedor constrói essa funcionalidade dentro da **camada de aplicação**.

1.5.2 O Processo de Encapsulamento

Fenômeno de Rede

Quando você clica em um link na Web, como o texto da sua requisição HTTP é transformado em impulsos elétricos ou luz sem perder o endereço de destino?

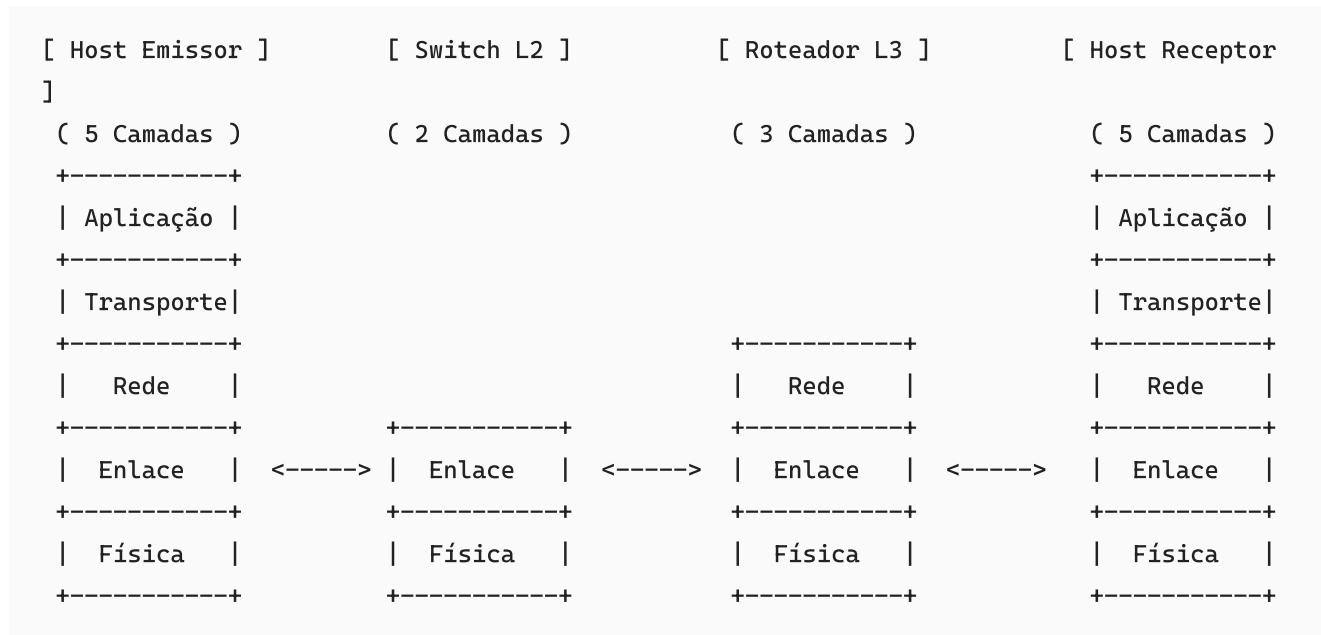
Conceito

Conforme os dados descem pela pilha de protocolos no sistema emissor, cada camada anexa suas próprias informações de controle na forma de um **cabeçalho** (*header*). O pacote completo da camada superior passa a ser a **carga útil** (*payload*) da camada inferior.

[Dados da Aplicação M]	--> Mensagem (Aplicação)
[Cabeçalho H_t Dados M]	--> Segmento (Transporte)
[Cabeçalho H_n Cabeçalho H_t Dados M]	--> Datagrama (Rede)
[Cabeçalho H_l Cabeçalho H_n H_t M Trailer T_l]	--> Quadro (Enlace)

Qual camada cada dispositivo da rede processa?

Nem todos os nós da rede implementam as 5 camadas da pilha:



1. **Sistemas Finais (Hosts):** Processam as **5 camadas** da pilha.
2. **Roteadores (Comutadores da Camada de Rede):** Processam até a **Camada 3 (Rede)**. Eles examinam o cabeçalho IP para tomar decisões de repasse, mas não alteram nem examinam a carga útil do segmento de transporte.
3. **Switches de Enlace (Camada 2):** Processam até a **Camada 2 (Enlace)**. Eles examinam endereços físicos (MAC) para repassar quadros dentro da mesma rede local.

1.6 Redes sob Ataque: Segurança de Redes

A Internet original foi projetada sob o modelo de um **grupo de usuários de confiança mútua** em uma rede transparente. Conforme se tornou a infraestrutura global da sociedade, surgiram diversas classes de ameaças maliciosas.

1.6.1 Malware: Infecção em Sistemas Finais

O *malware* é qualquer software malicioso que entra e infecta um sistema final.

- **Vírus:** *Malware* que exige alguma forma de interação do usuário (como executar um anexo de e-mail ou abrir um arquivo) para infectar o dispositivo.
 - **Worm (Verme):** *Malware* **autorreprodutivo** que busca ativamente outros sistemas vulneráveis na rede e se espalha de forma autônoma em velocidade exponencial, sem necessidade de ação humana.
 - **Botnet:** Uma rede de milhares de dispositivos infectados (chamados de *zumbis*) controlada remotamente por um atacante para disparar e-mails de *spam* ou ataques distribuídos de negação de serviço.
-

1.6.2 Ataques de Recusa de Serviço (DoS e DDoS)

Ataques de **Negação de Serviço** (*Denial-of-Service - DoS*) tornam uma rede, servidor ou infraestrutura inoperante para usuários legítimos. Dividem-se em três categorias:

1. **Ataque de Vulnerabilidade:** Envio de sequências de pacotes bem elaboradas a uma aplicação ou sistema operacional vulnerável para forçar uma falha ou parada do serviço.
2. **Inundação na Largura de Banda (*Bandwidth Flooding*):** O atacante envia uma quantidade maciça de pacotes ao servidor-alvo, tão alta que o enlace de acesso de taxa congestionada completamente, bloqueando pacotes legítimos.
3. **Inundação na Conexão (*Connection Flooding / SYN Flood*):** O atacante estabelece um grande número de conexões TCP semiabertas no servidor, esgotando sua memória e recursos de processamento.

DoS Distribuído (DDoS)

Se a taxa do enlace do servidor for muito grande, uma única fonte de ataque não consegue gerar tráfego suficiente para derrubá-lo. Em um **ataque DDoS**, o atacante utiliza uma **botnet** com milhares de zumbis para enviar tráfego simultâneo. A taxa agregada sobrecarrega a capacidade do alvo, tornando o ataque muito mais difícil de filtrar e conter.

1.6.3 Análise de Pacotes (*Packet Sniffing*)

Em meios de transmissão compartilhados (como redes Wi-Fi ou LANs Ethernet com hubs), um receptor passivo nas proximidades do transmissor pode obter uma cópia de todos os quadros e pacotes trafegados.

- **Características:** O analisador de pacotes (*packet sniffer*, como o software Wireshark) é passivo, ou seja, **não injeta pacotes** no canal, o que o torna extremamente difícil de ser detectado.
 - **Defesa Principal:** Utilização de **criptografia** (HTTPS, TLS/SSL, IPsec) em todas as camadas para impedir que a cópia lida seja compreendida pelo bisbilhoteiro.
-

1.6.4 Falsificação de Identidade (*IP Spoofing*)

Como a Internet não autentica nativamente o endereço de origem no cabeçalho do pacote IP no momento da criação, um atacante pode injetar na rede um pacote contendo um **endereço IP de origem falso**.

- **Consequências:** O receptor é enganado achando que a mensagem veio de uma entidade confiável e pode executar comandos maliciosos ou enviar respostas para a vítima do IP falsificado.
 - **Defesa:** Filtragem de entrada (*Ingress Filtering* - RFC 2827) configurada no roteador do primeiro salto para descartar pacotes cujo IP de origem não pertença à sub-rede física daquela interface.
-

1.7 História das Redes de Computadores e da Internet

A evolução da Internet pode ser dividida em quatro fases históricas marcantes:

1. 1961–1972: A Era Pioneira da Comutação de Pacotes

- **Bases Teóricas:** Leonard Kleinrock desenvolve a teoria de filas demonstrando a eficiência da comutação de pacotes em 1961; Paul Baran e Donald Davies propõem de forma independente redes chaveadas por pacotes.
- **ARPANET:** Em 1969, o primeiro nó da ARPANET é instalado na UCLA sob o patrocínio da DARPA. Em 1972, ocorre a primeira demonstração pública da ARPANET e o surgimento do primeiro protocolo de e-mail.

2. 1972–1980: Interconexão de Redes e Arquitetura Aberta

- **O Problema:** Redes independentes (ARPANET, ALOHANET via rádio, SATNET via satélite) não conseguiam se comunicar diretamente.
- **Surgimento do TCP/IP:** Vinton Cerf e Robert Kahn desenvolvem os princípios da arquitetura de **interconexão de redes (*interneting*)**, definindo o protocolo TCP/IP para unificar redes heterogêneas sob uma cintura fina comum.

3. 1980–1990: Proliferação de Redes e Padronização

- Em **1 de janeiro de 1983**, o TCP/IP é oficialmente adotado como o único protocolo padrão da ARPANET.
- Lançamento do **DNS (*Domain Name System*)** para substituir arquivos estáticos de mapeamento de nomes por um sistema hierárquico e distribuído.
- A **NSFNET** constrói um backbone de alta velocidade interconectando supercomputadores universitários, expandindo o acesso acadêmico.

4. 1990–Presente: A Era Comercial, a Web e a Internet Moderna

- **Invenção da World Wide Web:** Tim Berners-Lee inventa o HTML, HTTP, os servidores Web e os navegadores no CERN entre 1989 e 1991.
- **Privatização e Explosão Global:** Nos anos 90, a NSFNET desativa seu backbone comercial, dando lugar aos ISPs privados e à proliferação comercial.

- **A Era Contemporânea:** Expansão das redes móveis (4G/5G), aparecimento de redes de distribuição de conteúdo (CDNs), redes definidas por software (SDN), nuvens e novos protocolos de transporte rápido como **HTTP/3** e **QUIC**.
-

Capítulo 2: Camada de Aplicação

A camada de aplicação é o topo da pilha de protocolos e a interface direta com o usuário final e os programas de computador. Ela permite a criação de aplicações distribuídas que se comunicam através da infraestrutura de rede sem que o desenvolvedor precise conhecer os detalhes físicos das camadas inferiores.

2.1 Princípios das Aplicações de Rede

2.1.1 Arquiteturas de Aplicação: Cliente-Servidor vs. Peer-to-Peer (P2P)

Ao projetar uma aplicação de rede, a primeira decisão do desenvolvedor é definir a arquitetura de comunicação entre os sistemas finais:

1. Arquitetura Cliente-Servidor (*Client-Server*):

- **Servidor:** É um hospedeiro sempre ativo (*always-on*), com um endereço IP fixo e bem conhecido. Atende a requisições de potencialmente milhões de clientes. Para suportar altas cargas, os servidores são frequentemente organizados em *datacenters* ou fazendas de servidores.
- **Cliente:** É o processo que inicia a comunicação enviando uma requisição ao servidor. Os clientes podem ter endereços IP dinâmicos e não se comunicam diretamente entre si.
- **Gargalo:** Conforme o número de clientes cresce, o enlace de *upload* do servidor torna-se o gargalo, e o tempo de distribuição de dados cresce de forma linear.

2. Arquitetura Peer-to-Peer (P2P):

- **Funcionamento:** Não depende de servidores dedicados sempre ativos. Duplas de hospedeiros intermitentemente conectados (**pares** ou *peers*) comunicam-se diretamente entre si.
 - **Autoescalabilidade (*Self-scalability*):** Embora cada novo par traga carga adicional ao solicitar arquivos, ele também adiciona capacidade de serviço ao sistema ao redistribuir pedaços do arquivo para outros pares.
 - **Sessão P2P (Visão de Prova UFRJ):** Em uma aplicação P2P, a afirmação de que "*não existe noção de cliente e servidor*" é **falsa**. No contexto de uma sessão de comunicação individual entre dois pares, o processo que solicita o bloco age como **cliente** e o processo que envia o bloco age como **servidor**.
-

2.1.2 Comunicação entre Processos e Sockets

Um programa executado em um sistema final é um **processo**. Dois processos em *hosts* distintos comunicam-se trocando mensagens através da rede.

A Interface Socket

A comunicação entre a camada de aplicação e a camada de transporte no sistema operacional ocorre por meio de uma API chamada **socket**.

- **Analogia:** O socket é a "porta de uma casa". A aplicação entrega a mensagem no socket do seu lado; a infraestrutura de transporte abaixo se encarrega de levar a mensagem até o socket do destino.
- O desenvolvedor tem controle total sobre a camada de aplicação, mas seu controle sobre a camada de transporte resume-se a: (1) escolher o protocolo de transporte (TCP ou UDP) e (2) ajustar alguns parâmetros (como tamanho de *buffers* e portas).

Endereçamento de Processos

Para que uma mensagem atinja o processo correto em outro hospedeiro, são necessárias duas informações essenciais:

1. **Endereço IP (32 bits no IPv4 / 128 bits no IPv6):** Identifica univocamente o *host* de destino na rede global.
2. **Número de Porta (*Port Number*, 16 bits):** Identifica o processo/aplicação específico no *host* de destino.
 - Exemplo de portas padronizadas (*Well-Known Ports*):
 - **HTTP (Web):** Porta 80
 - **HTTPS (Web Segura):** Porta 443
 - **SMTP (E-mail):** Porta 25
 - **DNS (Nomes):** Porta 53

2.1.3 Requisitos de Transporte Exigidos pelas Aplicações

Diferentes aplicações possuem necessidades distintas quanto ao serviço fornecido pela camada de transporte:

1. **Confiabilidade e Integridade de Dados (*Data Loss*):**
 - *Aplicações tolerantes a perdas:* Áudio e vídeo em tempo real ou jogos *online* toleram uma pequena porcentagem de pacotes perdidos sem comprometer criticamente a experiência do usuário.
 - *Aplicações sensíveis à perda:* Transferência de arquivos (FTP), e-mail, transações bancárias e texto Web exigem entrega 100% confiável e sem erros.

2. Vazão (*Throughput*):

- *Aplicações sensíveis à largura de banda: Streaming* de vídeo em alta definição exige taxas de bits mínimas garantidas para evitar travamentos.
- *Aplicações elásticas:* E-mail, navegação Web e transferência de arquivos utilizam a largura de banda que estiver disponível no momento.

3. Temporização (*Timing / Latência*):

- Aplicações interativas em tempo real (telefonia IP, jogos multijogador) exigem garantias de baixos atrasos fim a fim (por exemplo, entrega de bits em menos de 100 ms) para evitar pausas artificiais na conversação.

4. Segurança:

- Criptografia de dados de ponta a ponta, integridade de mensagem e autenticação das partes comunicantes.

2.1.4 Serviços de Transporte Fornecidos pela Internet: TCP vs. UDP

A camada de transporte da Internet oferece dois protocolos principais para as aplicações:

PROPRIEDADE DE TRANSPORTE	TCP (<i>TRANSMISSION CONTROL PROTOCOL</i>)	UDP (<i>USER DATAGRAM PROTOCOL</i>)
Orientação à Conexão	Orientado à Conexão: Exige o <i>handshake</i> de três vias antes da troca de dados.	Sem Conexão (<i>Stateless</i>): Envia datagramas sem estabelecimento de conexão prévia.
Confiabilidade	Garantida: Entrega ordenada e sem perdas (<i>Byte-stream</i>) por ACKs e retransmissões.	Não Confiável: Pacotes podem ser perdidos, duplicados ou chegar fora de ordem.
Controle de Fluxo/Congestionamento	Sim: Adapta a taxa de envio para não afogar o receptor nem sobrecarregar os roteadores.	Não: Envia na taxa que a aplicação injetar no socket.
Garantia de Vazão/Delay	Não: Não oferece garantias mínimas de banda ou limite máximo de atraso.	Não: Não oferece garantias mínimas de banda ou limite máximo de atraso.
Sobrecarga (<i>Overhead</i>)	Maior (cabeçalho de 20 bytes + latência de <i>handshake</i>).	Mínima (cabeçalho leve de 8 bytes, transmissão imediata).
Uso Típico	Web (HTTP), E-mail (SMTP), Transferência de Arquivos.	DNS (consultas rápidas), Streaming de Vídeo/Áudio em Tempo Real, VoIP.

2.2 A Web e o HTTP

1. O Conceito de HTTP e a Semântica Sem Estado (*Stateless*)

Fenômeno de Rede

Quando um usuário navega pela Web, ele requisita arquivos HTML, imagens JPEG, arquivos CSS e scripts JS espalhados por servidores na rede.

Conceito

O HTTP (*Hypertext Transfer Protocol*) é o protocolo da camada de aplicação que governa a comunicação Web. Ele opera segundo a arquitetura **Cliente-Servidor**:

- **Navegador (Cliente):** Envia mensagens de requisição HTTP (*HTTP Request*) solicitando objetos.
- **Servidor Web:** Recebe a requisição, localiza o objeto e devolve uma mensagem de resposta (*HTTP Response*) contendo o objeto solicitado.

A Propriedade *Stateless* (Sem Estado)

O HTTP é um **protocolo sem estado (*stateless*)**. O servidor Web atende às requisições do cliente sem armazenar informações sobre requisições anteriores feitas por aquele mesmo usuário.

- **Vantagem:** Simplifica drasticamente o projeto do servidor, permitindo que ele atenda a milhões de requisições simultâneas sem consumir memória para manter o histórico de cada cliente.
 - **Contorno da Limitação:** Para aplicações que exigem estado (como carrinhos de compras e sessões de login), a camada de aplicação utiliza **cookies** e dados de sessão.
-

2. A Definição do RTT e o *Three-Way Handshake* do TCP

Para calcular quanto tempo leva para carregar uma página Web, precisamos quantificar o tempo de viagem de um pacote de ida e volta na rede.

Modelo Matemático: O RTT e a Abertura de Conexão TCP

1. O que representa?

- **RTT (*Round-Trip Time*):** O tempo necessário para um pequeno pacote viajar do cliente ao servidor e retornar ao cliente.
- **Composição do RTT:** O RTT inclui a soma de todos os atrasos físicos do caminho: atrasos de propagação nos enlaces, atrasos de fila nos roteadores e atrasos de processamento.

2. Por que estamos usando este modelo?

O HTTP não transporta dados diretamente pelo meio físico; ele utiliza o protocolo **TCP** na camada de transporte para garantir uma entrega confiável. Antes de enviar o primeiro `GET HTTP`, o cliente precisa obrigatoriamente estabelecer uma conexão TCP via **Apresentação de Três Vias (*Three-Way Handshake*)**.

3. Aplicação do Modelo de Conexão

1. **Passo 1 (SYN):** Cliente envia um segmento TCP SYN ao servidor.
2. **Passo 2 (SYN/ACK):** Servidor responde com um segmento TCP SYN/ACK. *(Estes dois passos consomem exatos 1 RTT).*
3. **Passo 3 (ACK + GET):** O cliente envia o ACK de confirmação combinado com a mensagem de requisição `HTTP GET`.
4. **Passo 4 (Resposta):** O servidor processa o pedido e envia o arquivo HTML/Objeto de volta. *(A requisição e a recepção do objeto consomem mais 1 RTT).*

4. Interpretação em Termos de Latência Web

A transferência do primeiro arquivo de uma página Web (o HTML base) exige, no mínimo:

2 RTT

onde 1 RTT é gasto no *handshake* TCP e 1 RTT é gasto na requisição/resposta HTTP.

3. Os 4 Modelos de Conexão HTTP e a Análise Matemática de Latência (Kurose + UFRJ)

Fenômeno de Rede

Uma página Web moderna raramente possui apenas um arquivo HTML; ela contém um arquivo HTML base e **objetos referenciados** (imagens, vídeos, scripts) no mesmo servidor. Como o navegador deve organizar o transporte desses objetos pela rede?

Combinando as opções de **Persistência** da conexão TCP com o **Paralelismo** do navegador, surgem 4 modelos clássicos de transporte analisados na UFRJ:

Modelo 1: HTTP Não Persistente Sem Paralelismo (HTTP/1.0 Padrão)

1. Funcionamento

A conexão TCP é aberta para transferir um único objeto e **imediatamente fechada** após a entrega. Os objetos referenciados são baixados estritamente em série.

2. Análise Matemática do Tempo Total

Para o arquivo-base HTML e os objetos referenciados:

- O HTML base custa (1 RTT para TCP + 1 RTT para HTTP).
- Cada um dos objetos exige uma nova conexão TCP, custando cada.

ãé

(desprezando o tempo de transmissão de objetos pequenos).

3. Avaliação de Desempenho

- **Vantagens:** O servidor libera memórias e descritores de *sockets* rapidamente assim que entrega o objeto, sendo mais simples e resiliente contra ataques de conexões presas.
- **Desvantagens:** Pior desempenho de latência. O cliente paga o custo de *handshake* repetidamente para cada imagem pequena da página.

Modelo 2: HTTP Não Persistente Com Paralelismo

1. Funcionamento

O navegador abre **múltiplas conexões TCP independentes e simultâneas** (geralmente até 6) para baixar os objetos em paralelo.

2. Análise Matemática do Tempo Total

- O HTML base custa 2 RTTs.
- Assumindo paralelismo perfeito (onde as conexões simultâneas baixam os objetos ao mesmo tempo), os RTTs dos objetos são sobrepostos.

ã

3. Avaliação de Desempenho

- **Vantagens:** Redução drástica no tempo de carregamento perceptível.
- **Desvantagens:** Consumo massivo de recursos no servidor, que precisa processar múltiplos *handshakes* e manter vários *sockets* abertos simultaneamente para o mesmo cliente.

Modelo 3: HTTP Persistente Sem Paralelismo (HTTP/1.1 Padrão / *Pipelining*)

1. Funcionamento

O servidor **deixa a conexão TCP aberta** após enviar a resposta do HTML. Os objetos subsequentes são solicitados através da **mesma conexão TCP** já estabelecida.

2. Análise Matemática do Tempo Total

- O HTML base custa (1 RTT TCP + 1 RTT HTTP).
- Como o TCP já está aberto, cada um dos objetos subsequentes custa apenas **1 RTT** (apenas o pedido/resposta HTTP).

é

3. Avaliação de Desempenho

- **Vantagens:** Economiza de *handshake* por objeto e poupa CPU e memória do servidor ao evitar a criação/destruição contínua de *sockets*.
- **Desvantagens:** O servidor precisa gerenciar *timeouts* de conexões ociosas. Continua sujeito ao bloqueio de cabeça de fila (*Head-of-Line Blocking*).

Modelo 4: HTTP Persistente Com Paralelismo (Web Moderna)

1. Funcionamento

O cliente **reaproveita a conexão TCP persistente existente** para enviar requisições sequenciais, mas **abre uma nova conexão TCP em paralelo** para acelerar objetos adicionais simultaneamente.

2. Análise Matemática do Tempo Total

- O HTML base e o objeto reusado na conexão 1 custam .
- A nova conexão paralela aberta para o objeto 2 custa .
- O tempo total da fase de objetos é o máximo entre as vias paralelas:

Tabela Resumo dos Modelos de Conexão HTTP (UFRJ)

MODELO DE CONEXÃO	CONEXÕES TCP ABERTAS	FÓRMULA DO TEMPO TOTAL (PARA OBJETOS)	VANTAGEM PRINCIPAL	DESVANTAGEM PRINCIPAL
Não Persistente Sem Paralelismo	conexões (1 por vez)		Liberação rápida de recursos no servidor.	Altíssima latência (paga RTT TCP para tudo).
Não Persistente Com Paralelismo	conexões (várias juntas)		Carregamento muito rápido.	Sobrecarga severa de CPU/sockets no servidor.

MODELO DE CONEXÃO	CONEXÕES TCP ABERTAS	FÓRMULA DO TEMPO TOTAL (PARA OBJETOS)	VANTAGEM PRINCIPAL	DESVANTAGEM PRINCIPAL
Persistente Sem Paralelismo	1 única conexão TCP		Economiza RTTs de TCP e recursos no servidor.	Sofre com Head-of-Line (HOL) Blocking.
Persistente Com Paralelismo	Múltiplas conexões mantidas		Equilíbrio entre paralelismo e reuso de TCP.	Alto número de conexões mantidas ociosas.

4. O Bloqueio de Cabeça de Fila (HOL Blocking) e a Evolução do HTTP (HTTP/1.1 HTTP/2 HTTP/3)

Fenômeno de Rede

No HTTP/1.1 com conexão persistente única, as requisições são atendidas na ordem de chegada (FCFS: *First-Come, First-Served*). Se o cliente solicitar um arquivo gigante (ex: um vídeo) seguido por três imagens pequenas, o que acontece?

Modelo Matemático: O Custo do Bloqueio HOL (UFRJ / Kurose)

1. Aplicação do Cálculo

Suponha que um objeto grande leve **10 unidades de tempo** para ser transmitido e três objetos pequenos () levem **1 unidade de tempo** cada.

- **Cenário A: Atendimento FCFS (Objeto Grande Primeiro - HTTP/1.1)**
 - Instante de conclusão de : .
 - Instante de conclusão de : .
 - Instante de conclusão de : .
 - Instante de conclusão de : .
 - **Tempo Médio de Conclusão Percebido:**
- **Cenário B: Atendimento Reordenado/Intercalado (Objetos Pequenos Primeiro)**
 - Instante de conclusão de : .
 - Instante de conclusão de : .
 - Instante de conclusão de : .
 - Instante de conclusão de : .
 - **Tempo Médio de Conclusão Percebido:**

2. Interpretação em Termos de Redes

O arquivo grande termina no exato mesmo instante () em ambos os cenários. No entanto, reordenar/intercalar a transmissão reduz o tempo médio de espera do usuário de **11,5 para 4,75 (uma redução de mais de 58%)** sem adicionar 1 bps de largura de banda à rede!

Solução Tecnológica nas Versões do HTTP

1. HTTP/2 (Padronizado em 2015):

- **Intercalação de Quadros (*Frame Interleaving*)**: Divide os objetos em quadros pequenos e intercala sua transmissão na mesma conexão TCP. O vídeo grande não bloqueia mais as imagens pequenas.
- **Problema Residual**: Como o HTTP/2 ainda roda sobre uma **única conexão TCP**, se um único pacote for perdido na rede, o TCP paralisa a entrega de **todos os quadros** até retransmitir o segmento perdido (Bloqueio HOL no nível de Transporte).

2. HTTP/3 (Padronizado via QUIC):

- Substitui o TCP subjacente pelo protocolo **QUIC (que opera sobre UDP)**.
 - O QUIC implementa fluxos de dados independentes e criptografados. Se um pacote de uma imagem for perdido, apenas aquele fluxo sofre retransmissão; todos os outros fluxos continuam fluindo sem nenhum bloqueio!
-

5. Caches Web (*Proxy Servers*) e o GET Condicional

Fenômeno de Rede

Como reduzir a latência percebida pelo usuário e evitar o estrangulamento de um enlace de acesso saturado sem precisar gastar fortunas aumentando a largura de banda física?

Conceito

Um **Cache Web (Servidor Proxy)** é um nó de rede instalado na LAN local que atende a requisições HTTP em nome do servidor de origem.

- **Dualidade Papel**: O cache age simultaneamente como **servidor** (para os navegadores locais) e como **cliente** (em relação aos servidores de origem na Internet).
-

Unificação Matemática: O Impacto do Cache no Atraso Médio (Notas UFRJ)

1. Aplicação do Cálculo

Considere uma rede institucional onde a taxa de acerto do cache (*Hit Rate*) é (40% das requisições são servidas localmente em). As 60% restantes () precisam atravessar a Internet e sofrem um atraso total de .

O **Atraso Médio Total de Resposta** é a média ponderada:

é

é

2. Interpretação em Termos de Redes

Além de cortar a latência média de mais de 2 segundos para 1,21 segundo, o cache reduz a carga no enlace de acesso em 40% (). Isso retira a intensidade de tráfego da zona de saturação (onde) e estabiliza as filas do roteador.

O Mecanismo do GET Condicional (304 Not Modified)

Para evitar que o cache entregue uma cópia local desatualizada de um arquivo que sofreu alterações no servidor de origem, o HTTP utiliza o **GET Condicional**:

```
[ Cliente ] ----> (GET /fig.png) ----> [ Cache Web ] --- (GET com If-Modified-Since) ---->
[ Servidor Origem ]
                                     [ Local ] <--- (304 Not Modified) ----- [
Servidor Origem ]
[ Cliente ] <--- (200 OK + Dados) -- [ Cache Web ]
```

1. Quando o cache armazena um objeto, ele salva a data enviada no cabeçalho **Last-Modified**: do servidor.
2. Quando outro cliente pede o mesmo objeto, o cache envia uma requisição de validação contendo o cabeçalho: **If-Modified-Since**: <Data da Cópia Local>
3. **Se o objeto NÃO foi modificado**: O servidor responde com o código de status **304 Not Modified** com um **corpo de mensagem completamente vazio**.
4. **Resultado**: Economiza-se 100% da largura de banda do arquivo e o cache entrega imediatamente a sua cópia local.

2.3 Correio Eletrônico na Internet: SMTP, POP3, IMAP e Webmail

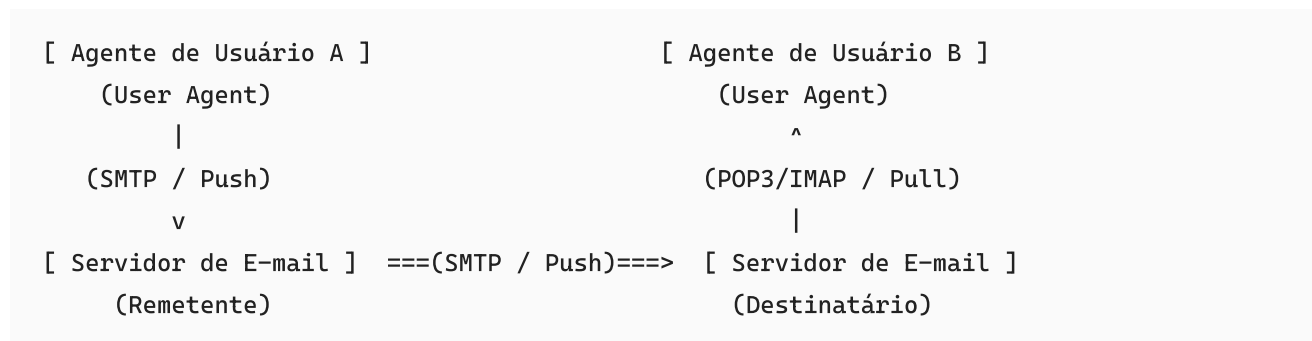
2.3.1 Arquitetura do Sistema de E-mail

Fenômeno de Rede

Ao contrário da navegação Web, onde a comunicação ocorre em tempo real entre o navegador do usuário e o servidor Web, o correio eletrônico é um sistema de **comunicação assíncrona**. O remetente envia a mensagem mesmo que o destinatário esteja *offline*.

Conceito

A arquitetura de e-mail é composta por três elementos fundamentais:



1. **Agentes de Usuário (User Agents - UA):** Softwares que permitem ao usuário ler, responder, criar e organizar mensagens (ex: Outlook, Thunderbird ou a interface do Gmail).
2. **Servidores de Correio (Mail Servers):** O coração da infraestrutura. Cada usuário possui uma **caixa de correio (mailbox)** localizada em seu servidor de e-mail.
 - **Fila de Mensagens (Message Queue):** Se o servidor do remetente não conseguir entregar uma mensagem imediatamente ao servidor do destinatário, a mensagem é mantida em uma fila para retentativas periódicas (ex: a cada 30 minutos). Se após alguns dias a entrega falhar, ela é devolvida com mensagem de erro (*bounce*).
3. **Protocolo SMTP (Simple Mail Transfer Protocol):** O protocolo padrão da camada de aplicação para transferência de e-mails entre servidores de correio.

2.3.2 O Protocolo SMTP vs. HTTP

Funcionamento do SMTP

- Opera sobre a camada de transporte usando **TCP na porta 25**.
- É um protocolo orientado a texto simples (comandos ASCII e códigos de resposta de 3 dígitos).
- A transferência de uma mensagem ocorre em três fases: **Handshake, Transferência de Dados e Encerramento**.
- O fim do corpo da mensagem é sinalizado por uma linha contendo estritamente um **ponto único (.)**.

Análise Comparativa: SMTP vs. HTTP (Tema Recorrente de Prova)

CRITÉRIO	HTTP	SMTP
Mecanismo de Comunicação	Protocolo <i>Pull</i> (Puxar): O cliente puxa/solicita dados hospedados no servidor.	Protocolo <i>Push</i> (Empurrar): O servidor do remetente empurra/envia os dados para o servidor do destinatário.
Estrutura de Dados	Cada objeto (texto, imagem, vídeo) é empacotado em sua própria mensagem de resposta HTTP .	Todos os componentes do e-mail são codificados e combinados em uma única mensagem composta .
Formato das Mensagens	Suporta nativamente dados binários e qualquer tipo de mídia no corpo.	Restrito historicamente ao formato ASCII de 7 bits (exigindo extensões MIME para anexos binários).
Porta Padrão TCP	Porta 80 (HTTP) / 443 (HTTPS).	Porta 25 .

2.3.3 Protocolos de Acesso ao Correio (*Mail Access Protocols*)

Fenômeno de Rede

Por que o destinatário não usa o SMTP para baixar os e-mails do seu próprio servidor de correio para o seu computador?

Conceito

O SMTP é um protocolo **exclusivamente *Push*** (utilizado para empurrar mensagens do remetente até o servidor do destino). Para puxar (*Pull*) os e-mails armazenados na sua caixa de correio no servidor para o seu dispositivo local, utilizam-se **Protocolos de Acesso ao Correio**:

1. POP3 (*Post Office Protocol - Version 3*):

- Protocolo extremamente simples (porta TCP 110).
- **Modo "Baixar e Apagar" (*Download-and-Delete*):** O agente de usuário baixa as mensagens para a máquina local e o servidor as apaga.
- **Modo "Baixar e Manter" (*Download-and-Keep*):** O usuário baixa as mensagens, mas mantém cópias no servidor.
- **Limitação:** É um protocolo **sem estado (*stateless*)** entre sessões. Se o usuário criar pastas ou marcar e-mails como lidos em uma máquina, essas alterações não se refletem em outros dispositivos.

2. IMAP (*Internet Message Access Protocol*):

- Protocolo muito mais complexo e robusto (porta TCP 143).
- **Mantém o Estado (*Stateful*):** Associa cada mensagem a uma pasta e mantém a árvore de diretórios centralizada no servidor.
- Permite que o usuário crie pastas, busque mensagens por palavra-chave no servidor e baixe apenas partes da mensagem (ex: ler o cabeçalho sem baixar um anexo de 50 MB). Ideal para

acesso por múltiplos dispositivos (*smartphone*, notebook, tablet).

3. Correio Baseado na Web (Webmail):

- O usuário utiliza um navegador Web comum (HTTP/HTTPS) para se comunicar com o servidor de e-mail.
- A comunicação entre o navegador e o servidor do provedor ocorre via **HTTP**.
- A comunicação entre os servidores de e-mail na Internet continua ocorrendo estritamente via **SMTP**.

2.4 DNS: O Serviço de Diretório da Internet (*Domain Name System*)

2.4.1 O que é o DNS e Por que Roda na Borda da Rede?

Fenômeno de Rede

Os seres humanos preferem identificar recursos na rede usando nomes mnemônicos e alfanuméricos fáceis de memorizar (como `www.ufrj.br`). No entanto, os roteadores no núcleo da rede processam pacotes examinando endereços IP numéricos de tamanho fixo (como `146.164.220.1`).

Conceito

O **DNS (*Domain Name System*)** é um banco de dados distribuído e hierárquico que provê a **tradução de nomes de domínio em endereços IP** (e vice-versa).

A Filosofia da Arquitetura do DNS

O DNS é um protocolo da **Camada de Aplicação**. Por que a função essencial de traduzir endereços de rede foi projetada na camada de aplicação e não dentro do núcleo da rede (na camada IP)?

- **Princípio de Projeto da Internet (*End-to-End Principle*)**: Manter o núcleo da rede (*routers*) o mais simples, rápido e enxuto possível, deixando a complexidade do banco de dados de nomes para os sistemas finais na borda (*edge*).

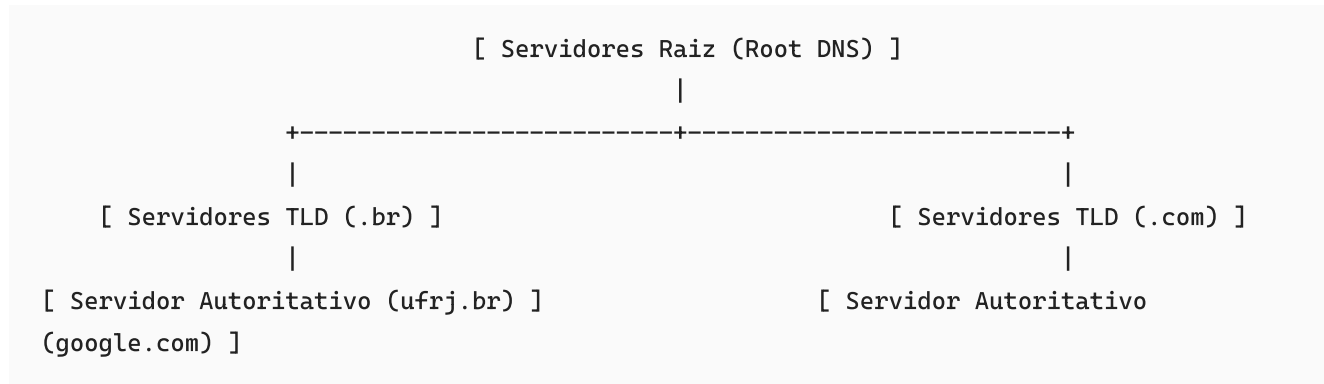
2.4.2 Arquitetura Hierárquica e Distribuída do DNS

Se o DNS fosse implementado como um **único servidor centralizado** no mundo, o sistema sofreria com:

1. **Ponto Único de Falha**: Se o servidor caísse, toda a Internet global parava.
2. **Volume Insuportável de Tráfego**: Bilhões de requisições por segundo congestionariam o enlace desse servidor.

3. **Distância Geográfica:** Consultas vindas do outro lado do mundo sofreriam com altíssimos atrasos de propagação.

Por isso, o DNS é estruturado como uma **árvore hierárquica distribuída**:



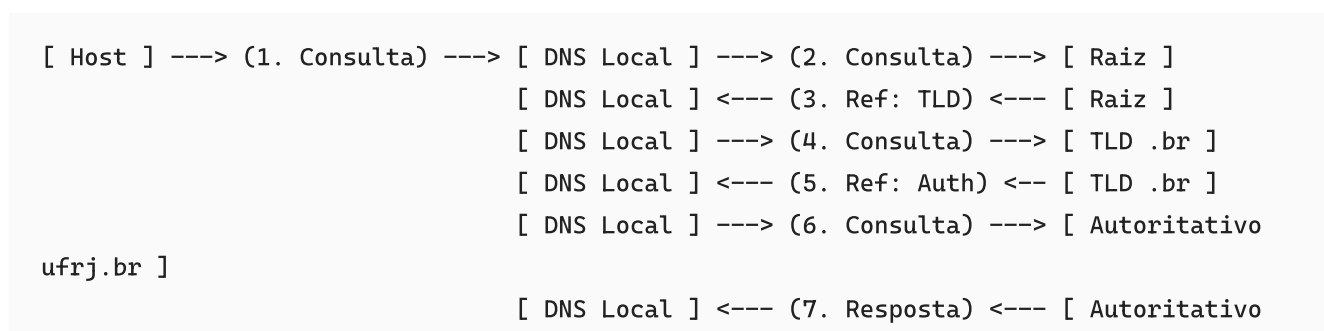
1. **Servidores Raiz (Root DNS Servers):** Existem 13 endereços IP lógicos de servidores raiz no mundo (replicados geograficamente por centenas de servidores físicos usando *Anycast*). Eles fornecem os endereços IP dos servidores TLD.
2. **Servidores de Domínio de Nível Superior (Top-Level Domain - TLD):** Responsáveis por domínios genéricos (`.com` , `.org` , `.edu`) e domínios de código de país (`.br` , `.uk` , `.fr`).
3. **Servidores Autoritativos (Authoritative DNS Servers):** Mantidos por organizações ou provedores. Contêm os registros DNS oficiais que mapeiam os nomes de hospedeiros daquela organização para seus IPs.
4. **Servidor DNS Local (Local Name Server / Resolver):** Não pertence estritamente à hierarquia, mas é fundamental. Cada ISP (ou rede universitária) possui um servidor DNS local. Quando um host faz uma consulta DNS, a requisição vai primeiramente para o seu DNS local, que atua como um intermediário/proxy resolvendo a cadeia.

2.4.3 Resolução de Nomes e Análise de RTTs: Iterativa vs. Recursiva (Kurose + UFRJ)

Fenômeno de Rede

Quando seu computador quer acessar `www.ufrj.br` , como o Servidor DNS Local descobre o IP final caso não o tenha em memória?

Exemplo com 4 RTTs em Resolução Iterativa.



```
ufrj.br ]  
[ Host ] <--- (8. IP Final) ---- [ DNS Local ]
```

Unificação Matemática: Modelo de Latência por RTTs na Resolução DNS

1. Resolução Iterativa (Padrão da Internet)

Na consulta **iterativa**, o servidor consultado responde com o endereço do próximo servidor da hierarquia que deve ser consultado (*"Eu não sei o IP, mas pergunte ao servidor X"*).

- **Análise de RTTs:**
 - Host DNS Local: **1 RTT** (geralmente na rede local).
 - DNS Local Servidor Raiz: **1 RTT**.
 - DNS Local Servidor TLD (`.br`): **1 RTT**.
 - DNS Local Servidor Autoritativo (`ufrj.br`): **1 RTT**.
- **Atraso Total sem Cache:**

2. Resolução Recursiva

Na consulta **recursiva**, o nó consultado assume a responsabilidade de contatar o nó seguinte e só devolve a resposta quando obtiver o IP final.

- **Carga:** Sobrecarrega os servidores dos níveis superiores da hierarquia (especialmente os servidores Raiz), e por isso é **desabilitada por padrão** nos servidores raiz e TLDs por questões de segurança e desempenho.

3. Ponto-chave de prova: por que ITERATIVO é melhor para POPULAR o cache do servidor local

Essa é uma pergunta que aparece repetidamente nas provas antigas (2026/1 Q1d, 2025/2 Q4a): **entre o host final e o servidor DNS local, sempre se usa resolução recursiva** (o host tem poucos recursos e delega tudo). A pergunta interessante é sobre a etapa seguinte: **entre o servidor local e o resto da hierarquia (raiz/TLD/autoritativo)**, qual abordagem é melhor?

- Se o servidor local usar **recursiva** nessa etapa (delegando a um outro resolver), ele **só recebe a resposta final** — não vê nem armazena nenhuma informação intermediária.
- Se o servidor local usar **iterativa**, ele mesmo vai "saltando" de servidor em servidor, e **em cada resposta parcial** ("pergunte ao servidor X") ele pode **armazenar em cache** aquele registro (por exemplo, o endereço do servidor TLD `.br`, ou o servidor NS autoritativo de `ufrj.br`). Assim, na **próxima** consulta a um domínio diferente mas sob o mesmo TLD, o servidor local **já sabe** para qual TLD ir, sem precisar consultar a raiz de novo.

Conclusão para a prova: a abordagem **iterativa favorece mais o preenchimento do cache do servidor local**, porque ele enxerga e armazena cada passo intermediário da resolução — a recursiva

"esconde" essa informação intermediária dentro do outro resolver.

2.4.4 Registros de Recursos (*Resource Records - RRs*)

O banco de dados do DNS armazena suas informações em registros de quatro campos no formato:

O campo **Type** define o significado dos campos **Name** e **Value** :

1. **Tipo A (*Address*)**: Mapeia um nome de host para um endereço IPv4.
 - (aulas.ufrj.br, 146.164.220.1, A, 86400)
 2. **Tipo NS (*Name Server*)**: Especifica o nome do servidor DNS autoritativo responsável pelo domínio.
 - (ufrj.br, dns.ufrj.br, NS, 86400)
 3. **Tipo CNAME (*Canonical Name*)**: Mapeia um alias/apelido para seu nome canônico (verdadeiro).
 - (server1.ufrj.br, webserver-prod-01.ufrj.br, CNAME, 86400)
 4. **Tipo MX (*Mail Exchange*)**: Mapeia o domínio para o nome do servidor de correio responsável por receber e-mails daquele domínio.
 - (ufrj.br, mail.ufrj.br, MX, 86400)
-

2.4.5 Caching DNS e a Escolha do Transporte: UDP vs. TCP (UFRJ)

Caching DNS

Para reduzir drasticamente o tempo de resposta e o tráfego nos servidores Raiz/TLD, os servidores DNS locais armazenam em **cache** os mapeamentos aprendidos.

- O campo **TTL (*Time to Live*)** especifica após quantos segundos a entrada expira e deve ser descartada do cache para garantir a consistência de dados alterados.
-

Unificação Matemática: Por que o DNS usa UDP por padrão e TCP como Exceção? (Tema Chave de Prova)

1. Uso do UDP (Porta 53) — Consultas Padrão

- **Fenômeno**: Uma consulta DNS típica (pergunta por um IP) exige uma mensagem pequena de requisição e uma resposta enxuta.
- **Análise de Latência**:
 - Se o DNS utilizasse **TCP**, cada tradução de nome exigiria o *handshake* de 3 vias do TCP (1 RTT) antes mesmo de enviar a pergunta DNS. A resolução DNS sem cache custaria no mínimo !

- Com **UDP**, o cliente envia o datagrama direto. A consulta custa apenas **1 RTT por salto**, tornando a navegação Web instantânea.
- **Economia de Recursos:** Servidores DNS recebem bilhões de requisições. Usar UDP evita manter blocos de controle de conexão TCP (*Sockets*) em memória no servidor.

2. Uso do TCP (Porta 53) — Exceção / Transferência de Zona

- **Fenômeno:** Quando a resposta DNS excede o tamanho máximo de um datagrama UDP padrão (**512 bytes**) ou quando dois servidores autoritativos sincronizam todo o seu banco de dados (**Transferência de Zona / Zone Transfer**).
- **Por que mudar para TCP?** Transferir bancos de dados inteiros exige a garantia de entrega confiável, sem perdas, sem corrupção e com ordenação correta, justificando o uso do TCP. Vamos concluir o **Capítulo 2: Camada de Aplicação**, cobrindo as Seções **2.5 (Distribuição P2P)**, **2.6 (Streaming de Vídeo DASH e CDNs)** e **2.7 (Programação com Sockets)**.

Sempre que a modelagem matemática e os exercícios de prova da UFRJ aprofundarem o texto do Kurose & Ross, unificaremos o conteúdo através da nossa cadeia analítica (**fenômeno de rede → conceito → modelo → matemática → cálculo → interpretação**).

2.5 Distribuição de Arquivos P2P e o Protocolo BitTorrent

1. O Fenômeno da Distribuição Massiva de Arquivos

Fenômeno de Rede

Quando uma empresa lança uma atualização de sistema operacional ou um vídeo em alta definição de **bits** e **usuários** tentam baixá-lo ao mesmo tempo, como a infraestrutura de rede responde?

Conceito: Arquitetura Cliente-Servidor vs. P2P

- **Na Arquitetura Cliente-Servidor:** O servidor é a única fonte geradora de bits. Conforme o número de clientes cresce, a taxa de upload do servidor () torna-se um gargalo severo.
 - **Na Arquitetura Peer-to-Peer (P2P):** Os sistemas finais (**pares** ou *peers*) atuam simultaneamente como clientes (consumindo bits) e servidores (redistribuindo bits que já baixaram). O sistema possui **autoescalabilidade (self-scalability)**: cada novo par traz uma nova carga de consumo, mas também adiciona capacidade de upload ao sistema.
-

2. Unificação Matemática: Modelo do Tempo Mínimo de Distribuição (Kurose + UFRJ)

Para comparar quantitativamente o tempo necessário para distribuir um arquivo de tamanho para clientes entre as duas arquiteturas, define-se o modelo sob as seguintes variáveis:

- : Tamanho do arquivo a ser distribuído (em **bits**).
 - : Número de clientes/pares que desejam obter a cópia do arquivo.
 - : Taxa de upload do servidor de origem (em **bits/s**).
 - : Taxa de upload do -ésimo par (em **bits/s**).
 - : Taxa de download do -ésimo par (em **bits/s**).
 - : Taxa de download do par mais lento da rede.
-

Modelo Matemático 1: Tempo de Distribuição na Arquitetura Cliente-Servidor ()

1. Dedução dos Limites Inferiores

Nenhum par ajuda a redistribuir o arquivo. O tempo total é limitado por dois gargalos físicos:

1. **Gargalo no Servidor:** O servidor precisa enviar cópias completas de bits, enviando um total de bits pela sua interface de upload . O tempo não pode ser menor que .
2. **Gargalo no Cliente Mais Lento:** O cliente com a menor taxa de download () leva no mínimo para receber seus próprios bits.

2. Aplicação da Fórmula

3. Interpretação em Termos de Redes

Para valores grandes de , o tempo de distribuição é dominado pelo termo . **O tempo cresce de forma estritamente linear com o número de usuários** . Se o número de clientes aumentar 1000 vezes, o tempo para distribuir o arquivo também aumentará 1000 vezes!

Modelo Matemático 2: Tempo de Distribuição na Arquitetura P2P ()

1. Dedução dos Limites Inferiores

No P2P, os pares redistribuem pedaços do arquivo entre si. O tempo total é limitado por três restrições físicas:

1. **Envio Inicial do Servidor:** Para que o arquivo entre na comunidade, o servidor precisa injetar cada um dos bits pelo menos uma vez no enlace. Tempo mínimo: .
2. **Gargalo de Download no Cliente Lento:** O cliente mais lento ainda precisa baixar seus bits. Tempo mínimo: .
3. **Capacidade Agregada de Upload do Sistema:** A rede como um todo precisa entregar um total de bits para os clientes. A taxa máxima de upload combinada de todo o sistema é a soma do upload

do servidor com o upload de todos os pares (). Tempo mínimo: .

2. Aplicação da Fórmula

3. Interpretação em Termos de Redes (Análise com Uploads Iguais)

Se todos os pares tiverem a mesma taxa de upload , a capacidade total de upload torna-se . O terceiro termo passa a ser:

Quando , dividindo o numerador e o denominador por :

O tempo de distribuição P2P **não cresce indefinidamente com** ; ele atinge uma assíntota e fica limitado superiormente! O gráfico de vs. curva-se e estabiliza, demonstrando a **autoescalabilidade do P2P**.

3. O Protocolo BitTorrent: Mecanismos Práticos de Funcionamento

O BitTorrent é o protocolo P2P de distribuição de arquivos mais popular do mundo.

- **Torrent e Chunks:** A coleção de todos os pares compartilhando um arquivo é chamada de **torrent**. O arquivo é dividido em blocos idênticos chamados **chunks** (tipicamente de **256 KB**).
 - **Rastreador (*Tracker*):** Nó central de infraestrutura que mantém o registro de quais pares estão ativos no torrent. Quando um novo par (Alice) entra na rede, ela se registra no *tracker* e recebe uma lista com o endereço IP de um subconjunto de pares (seus "vizinhos").
 - **Seleção de Chunks — O Mais Raro Primeiro (*Rarest First*):** Para decidir qual bloco pedir aos seus vizinhos, Alice determina quais blocos possuem o menor número de cópias disponíveis entre eles e **solicita os blocos mais raros primeiro**. Isso equaliza o número de cópias de cada bloco na rede e evita que um bloco desapareça se a fonte original sair.
 - **Algoritmo de Incentivo — *Tit-for-Tat* (Olho por Olho):** Como evitar que usuários "caronas" (*freeriders*) apenas baixem arquivos sem enviar nada em troca?
 1. Alice mede continuamente a taxa na qual recebe dados de cada vizinho.
 2. Ela seleciona os **4 pares que lhe fornecem dados na maior taxa** e retribui enviando blocos para eles. Esses 4 pares são chamados de **desbloqueados (*unchoked*)**. A lista é recalculada a cada 10 segundos.
 3. **Desbloqueio Otimista (*Optimistically Unchoked*):** A cada 30 segundos, Alice escolhe aleatoriamente **1 par adicional** (Bob) e envia blocos para ele. Se Bob retribuir com uma taxa alta, ele pode entrar na lista dos "Top 4" de Alice no ciclo seguinte. Isso permite que novos pares sem blocos consigam suas primeiras peças e que pares com altas capacidades de upload se encontrem.
-

2.6 Streaming de Vídeo e Redes de Distribuição de Conteúdo (CDNs)

2.6.1 O Desafio do Streaming de Vídeo e o Protocolo DASH

Fenômeno de Rede

O tráfego de vídeo (YouTube, Netflix, Prime Video) representa cerca de **80% de todo o tráfego da Internet**. Um vídeo pré-gravado é uma sequência de imagens exibidas a uma taxa fixa (ex: 24 ou 60 quadros/s). Como entregar vídeo contínuo para usuários com conexões oscilantes (ex: 4G/5G em movimento)?

Conceito: DASH (*Dynamic Adaptive Streaming over HTTP*)

No streaming tradicional por HTTP, o vídeo era baixado como um arquivo único. No **DASH**, o vídeo é codificado em **múltiplas versões de qualidade/bitrate** e dividido em trechos (*chunks*) de alguns segundos de duração (ex: 2 a 10 segundos).

O Arquivo de Manifesto (*Manifest File*)

O servidor HTTP fornece um **arquivo de manifesto** que lista as URLs e as taxas de bits de cada versão de qualidade do vídeo.

1. O cliente baixa primeiramente o arquivo de manifesto.
2. Em seguida, a aplicação cliente requisita um trecho de vídeo por vez via requisições `HTTP GET` especificando o intervalo de bytes.
3. **Seleção Adaptativa de Taxa:** Conforme baixa os trechos, a aplicação cliente mede a largura de banda de recepção atual e monitora o nível de preenchimento do seu *buffer* local. Se a rede acelerar, ela pede o próximo trecho em alta definição (4K); se o *buffer* esvaziar ou a rede oscilar, ela alterna dinamicamente para uma versão de menor taxa de bits (720p/480p), evitando travamentos na exibição.

2.6.2 Redes de Distribuição de Conteúdo (*Content Distribution Networks - CDNs*)

Fenômeno de Rede

Se uma empresa de streaming mantiver um único *datacenter* gigante com todos os seus vídeos, clientes distantes sofrerão com altos atrasos de propagação, travamentos em enlaces gargalo e o *datacenter* será um ponto único de falha.

Conceito

Uma **CDN** é uma rede geograficamente distribuída de servidores *proxy* que armazena cópias de vídeos e conteúdos em locais próximos aos usuários finais.

Filosofias de Posicionamento de Servidores de CDN

1. **Entrar Fundo (*Enter Deep*):** Instala pequenos *clusters* de servidores profundamente **dentro das redes de acesso dos ISPs residenciais**.
 - *Vantagem:* Minimiza a latência e contorna os gargalos da Internet pública.
 - *Desvantagem:* Alta complexidade de manutenção e gerenciamento de milhares de *clusters* espalhados.
2. **Trazer para Perto (*Bring Home*):** Instala *clusters* maiores em grandes **Pontos de Troca de Tráfego (IXPs)** interconectando ISPs de Nível 1/2.
 - *Vantagem:* Menor número de *clusters* para manter.
 - *Desvantagem:* Resulta em latências ligeiramente maiores do que o modelo *Enter Deep*.

Estratégias de Atualização de Conteúdo

- **Caches sob Demanda (*Pull-Caching*):** Se um cliente pede um vídeo que não está no *cluster* local da CDN, o servidor busca o vídeo no repositório central, armazena uma cópia e o entrega ao cliente (usado pelo Google/YouTube).
- **Armazenamento Agendado (*Push-Caching*):** Conteúdos e filmes populares são enviados ativamente para os servidores da CDN em horários agendados fora do horário de pico (estratégia utilizada pela Netflix).

2.7 Programação com Sockets em Python: UDP vs. TCP

Para encerrar o Capítulo 2, analisaremos como as aplicações interagem com os protocolos de transporte através da **Socket API**.

2.7.1 Sockets UDP em Python (Sem Conexão)

Conceito

No UDP, **não há fase de estabelecimento de conexão** (*handshake*). O remetente anexa explicitamente o endereço IP e a porta de destino a cada datagrama enviado.

O Servidor UDP e o Número de Sockets (Questão de Prova)

Um servidor UDP precisa de **apenas 1 único socket** para atender a qualquer número de clientes simultâneos! Como o UDP não mantém estado de conexão, todas as mensagens de todos os clientes entram pela mesma "porta" do socket do servidor, e o servidor identifica quem enviou examinando a tupla (IP_cliente, Porta_cliente) retornada pela primitiva de recepção.

Código Prático UDP em Python

```
# --- SERVIDIOR UDP (UDPServer.py) ---
from socket import *

serverPort = 12000
# Cria o socket UDP (AF_INET = IPv4, SOCK_DGRAM = UDP)
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort)) # Associa o socket à porta 12000
print("Servidor UDP pronto para receber...")

while True:
    message, clientAddress = serverSocket.recvfrom(2048) # Recebe dados e endereço do
    cliente
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress) # Responde ao cliente

# --- CLIENTE UDP (UDPClient.py) ---
from socket import *

serverName = '127.0.0.1' # IP do Servidor
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM) # Cria socket UDP do cliente

message = "mensagem de teste"
clientSocket.sendto(message.encode(), (serverName, serverPort)) # Envia datagrama
modifiedMessage, serverAddress = clientSocket.recvfrom(2048) # Recebe resposta
print("Resposta do Servidor:", modifiedMessage.decode())
clientSocket.close()
```

2.7.2 Sockets TCP em Python (Orientado à Conexão)

Conceito

No TCP, antes de trocar dados, cliente e servidor executam o *handshake* de 3 vias. Uma vez estabelecida, a comunicação ocorre como um **fluxo contínuo de bytes (byte-stream)**.

Por que o Servidor TCP precisa de 2 Tipos de Sockets? (Questão Chave de Prova)

Diferente do UDP, um servidor TCP utiliza **dois tipos de sockets**:

1. **Socket de Boas-Vindas (Welcome Socket / Listening Socket)**: Fica associado à porta bem conhecida (ex: porta 12000) escutando requisições de conexão de novos clientes (`listen()`).
 2. **Socket de Conexão (Connection Socket)**: Quando um cliente inicia um *handshake*, a chamada `accept()` do servidor cria um **novo socket dedicado exclusivamente** para conversar com aquele cliente específico.
- **Interpretação e Cálculo**: Se um servidor TCP estiver atendendo a **clientes simultâneos**, ele manterá abertos exatamente **sockets** em memória (1 socket de boas-vindas escutando a porta

principal + sockets de conexão dedicados aos clientes ativos).

Código Prático TCP em Python

```
# --- SERVIDOR TCP (TCPServer.py) ---
from socket import *

serverPort = 12000
# 1. Cria o Socket de Boas-Vindas (AF_INET = IPv4, SOCK_STREAM = TCP)
welcomeSocket = socket(AF_INET, SOCK_STREAM)
welcomeSocket.bind(('', serverPort))
welcomeSocket.listen(1) # Começa a escutar requisições TCP
print("Servidor TCP escutando na porta 12000...")

while True:
    # 2. accept() bloqueia até chegar um cliente e CRIA o Socket de Conexão dedicado
    connectionSocket, addr = welcomeSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())

    connectionSocket.close() # Fecha o socket de conexão do cliente atual

# --- CLIENTE TCP (TCPClient.py) ---
from socket import *

serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)

# Inicia o Three-Way Handshake TCP com o servidor
clientSocket.connect((serverName, serverPort))

sentence = "mensagem de teste tcp"
clientSocket.send(sentence.encode()) # Envia fluxo de bytes
modifiedSentence = clientSocket.recv(1024).decode()
print("Resposta do Servidor:", modifiedSentence)
clientSocket.close()
```

Tabela Resumo: Programação com Sockets UDP vs. TCP

PROPRIEDADE	SOCKETS UDP	SOCKETS TCP
Tipo de Socket Python	SOCK_DGRAM	SOCK_STREAM
Estabelecimento de Conexão	Não há (connect opcional).	Obrigatório via connect() e accept() .
Primitivas de E/S	sendto() e recvfrom() (com endereçamento explícito).	send() e recv() (pelo canal já estabelecido).

PROPRIEDADE	SOCKETS UDP	SOCKETS TCP
Abstração de Dados	Datagramas discretos (preserva limites de mensagem).	Fluxo contínuo de bytes (<i>Byte-Stream</i>).
Sockets no Servidor (para clientes)	1 socket único para todos os clientes.	sockets (1 de boas-vindas + de conexão).
Ordem de Execução dos Programas	O cliente pode enviar mensagens antes do servidor rodar (dados serão perdidos).	O programa servidor precisa rodar antes para abrir o socket e escutar na porta.

Capítulo 3 (parcial — Seções 3.1 a 3.5): Camada de Transporte — TCP e UDP

Nota de escopo: esta seção cobre exatamente o que foi confirmado como matéria de prova — estrutura e funcionamento básico de TCP e UDP. **Controle de congestionamento (Seções 3.6-3.7) está fora do escopo** e não é tratado aqui.

3.1 Introdução à Camada de Transporte

Fenômeno de Rede

A camada de **rede** (IP) só sabe entregar dados **de um host até outro host**, usando o endereço IP. Mas um host roda **vários processos** simultaneamente (navegador, e-mail, jogo online), todos compartilhando o mesmo IP.

Conceito

A camada de **transporte** é responsável pela "última entrega": pegar os dados que chegaram até o host (via IP) e encaminhá-los para o **processo correto**, usando o número de porta. Isso é o processo de **demultiplexação**. O processo inverso, no lado de quem envia — juntar dados de vários processos diferentes e passar para a camada de rede — é a **multiplexação**.

3.2 Multiplexação e Demultiplexação — Formalização

Como o sistema operacional identifica de quem é cada segmento

Cada segmento de transporte carrega, no seu cabeçalho, no mínimo dois campos: **porta de origem** e **porta de destino**.

Demultiplexação sem conexão (UDP)

Um socket UDP é identificado apenas por uma **dupla**: (endereço IP de destino, porta de destino). Isso significa que **dois segmentos UDP vindos de IPs de origem diferentes, mas com o mesmo IP e porta de destino**, são entregues ao **mesmo socket** — o UDP não diferencia origens diferentes, ele só olha para onde o segmento está indo.

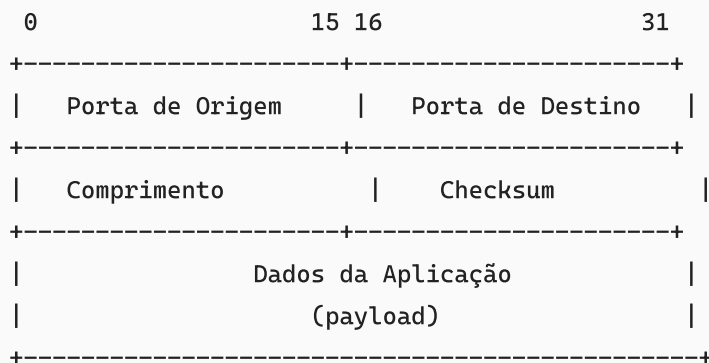
Demultiplexação orientada a conexão (TCP)

Um socket TCP é identificado por uma **quádrupla**: (IP de origem, porta de origem, IP de destino, porta de destino). Isso significa que o sistema operacional consegue diferenciar **conexões diferentes chegando na mesma porta de destino** — por exemplo, dois navegadores diferentes no seu computador, ambos acessando o mesmo site (mesmo IP:porta de destino), geram **quádruplas diferentes** (portas de origem diferentes, escolhidas aleatoriamente pelo sistema operacional para cada conexão nova), e por isso o servidor consegue tratar cada conexão de forma isolada, com seu próprio `connectionSocket` dedicado.

Por isso o servidor Web consegue atender múltiplos clientes simultâneos na mesma porta 80: cada cliente gera uma quádrupla única, mesmo compartilhando IP e porta de destino.

3.3 UDP — Estrutura e Funcionamento

Formato do segmento UDP (cabeçalho de apenas 8 bytes)



- **Porta de origem / destino**: 16 bits cada — usadas na demultiplexação.
- **Comprimento**: tamanho do segmento UDP (cabeçalho + dados), em bytes.
- **Checksum**: verifica se houve erro de bit durante a transmissão — soma de verificação simples, calculada sobre o segmento inteiro (cabeçalho + dados) mais um "pseudo-cabeçalho" com os IPs de origem/destino.

Por que UDP é "leve"

UDP **não** estabelece conexão, **não** garante entrega, **não** garante ordem, e **não** faz controle de fluxo nem de congestionamento. Isso o torna ideal para aplicações que preferem velocidade/simplicidade

a garantias (DNS, VoIP, streaming ao vivo, jogos), como já discutimos no Capítulo 2.

3.4 Transferência Confiável de Dados (RDT) — Construindo do Zero

Essa é a base conceitual de como o TCP garante entrega confiável **por cima** de uma rede que, por si só (a camada IP), **não** garante nada. O livro constrói isso incrementalmente, em versões (rdt 1.0, 2.0, 2.1, 2.2, 3.0) — cada uma resolvendo um problema novo que a versão anterior não cobria.

rdt 1.0 — Canal perfeitamente confiável (caso trivial)

Se o canal de comunicação **nunca** perde, corrompe ou atrasa nada, o transmissor simplesmente envia, e o receptor simplesmente recebe. Nenhum mecanismo extra é necessário. Isso é irrealista, mas serve de ponto de partida.

rdt 2.0 — Canal com erros de bit (mas sem perdas)

Agora os bits podem ser **corrompidos** durante a transmissão (mas não perdidos). O receptor precisa de uma forma de **detectar** erros, e o transmissor precisa saber se deve **retransmitir**.

Mecanismos introduzidos:

- **Checksum:** permite ao receptor detectar corrupção.
- **ACK (Acknowledgment):** o receptor envia de volta "recebi certo" quando o pacote chega sem erro.
- **NAK (Negative Acknowledgment):** o receptor envia "recebi errado" quando detecta corrupção via checksum — o transmissor então retransmite o mesmo pacote.

Problema não resolvido por rdt 2.0: e se o **próprio ACK/NAK** for corrompido? O transmissor não saberia se deve retransmitir ou não.

rdt 2.1 — Resolvendo ACK/NAK corrompidos com números de sequência

Solução: adicionar um **número de sequência** de 1 bit (0 ou 1, alternando a cada novo pacote — por isso esse protocolo também é chamado de **protocolo do bit alternante**, *alternating bit protocol*) a cada pacote de dados.

Se o transmissor recebe um ACK/NAK **corrompido** (ilegível), ele simplesmente **retransmite** o **último** pacote enviado (por segurança, já que não sabe se foi bem recebido). O receptor, ao receber um pacote **duplicado** (mesmo número de sequência do último que já recebeu com sucesso), **descarta** os dados (já os processou antes) mas **reenvia o ACK** — para garantir que o transmissor eventualmente saiba que está tudo certo.

Pergunta clássica de prova: "Quando o emissor recebe um ACK corrompido ou inesperado, o que ele deve fazer?" — **Resposta:** ele **ignora** esse ACK (não sabe interpretá-lo com confiança) e **aguarda o timeout**; se o timeout expirar sem receber um ACK válido, ele **retransmite** o último pacote enviado.

rdt 2.2 — Versão só com ACKs (sem NAKs)

É possível eliminar o NAK completamente: em vez de mandar "NAK, está corrompido", o receptor manda um **ACK duplicado** (reenvia o ACK do **último pacote correto recebido**). Quando o transmissor recebe **dois ACKs iguais em seguida** (um ACK duplicado), ele interpreta isso como um sinal implícito de que o próximo pacote esperado não chegou corretamente, e retransmite.

Pergunta clássica de prova: *"É possível implementar comunicação confiável usando apenas ACKs, sem NAKs?"* — **Resposta: Sim.** ACKs duplicados cumprem exatamente o mesmo papel informativo que um NAK explícito.

rdt 3.0 — Canal com erros E perdas (o protocolo "Stop-and-Wait" completo)

Agora, além de corrupção, o canal também pode **perder** pacotes ou ACKs inteiros (não chegam de jeito nenhum). Como o transmissor detecta uma perda, se nunca vai receber nenhum ACK/NAK sobre ela?

Solução: temporizador (timeout). O transmissor, toda vez que envia um pacote, **inicia um timer**. Se o ACK correspondente não chegar antes do timer expirar, o transmissor **assume que houve perda** (do pacote ou do ACK) e **retransmite**.

O dilema do valor do timeout (pergunta recorrente de prova):

- **Timeout muito pequeno:** o transmissor retransmite **antes da hora**, mesmo quando o pacote/ACK só está **atrasado** (ainda a caminho) — gerando retransmissões desnecessárias e desperdício de banda.
- **Timeout muito grande:** em caso de perda real, o transmissor **demora demais** para perceber e retransmitir — desperdiçando tempo ocioso e reduzindo a vazão efetiva.
- **Solução do TCP (avançado, mas cobrado):** o timeout é calculado **dinamicamente**, baseado numa **média móvel ponderada do RTT observado** (EstimatedRTT) mais uma margem de segurança proporcional à **variação** do RTT (DevRTT) — não é um valor fixo.

O que o receptor faz ao receber um pacote com número de sequência inesperado (pergunta recorrente): o receptor **descarta silenciosamente** o pacote errado (fora de ordem) e **reenvia o ACK do último pacote recebido corretamente em ordem** — isso "empurra" o transmissor a perceber, via ACKs duplicados ou timeout, que precisa retransmitir o que falta.

Utilização (eficiência) do protocolo Stop-and-Wait

Como o transmissor **espera** o ACK antes de enviar o próximo pacote, o enlace fica **ocioso** durante boa parte do tempo — especialmente se o RTT for grande comparado ao tempo de transmissão. A **utilização** U é definida como a fração de tempo em que o enlace está efetivamente ocupado transmitindo dados:

onde t é o tempo de transmissão de um pacote de dados, e RTT é o tempo de propagação de ida e volta (assumindo tempo de transmissão do ACK desprezível). **Isso é literalmente o mesmo cálculo que já vimos no gabarito de 2025/2 (Questão 1d):** com (só 1 número de sequência de espaço, janela

), o protocolo se comporta exatamente como stop-and-wait, e a utilização é essa fração — a mesma fórmula de "tempo útil ÷ tempo total do ciclo" que já vimos em outros contextos (como intensidade de tráfego).

Interpretação: para RTTs grandes (enlaces de longa distância, como satélite), a utilização do stop-and-wait pode ser **catastroficamente baixa** — isso motiva a próxima técnica.

Pipelining — Go-Back-N (GBN) e Selective Repeat (SR)

O problema do stop-and-wait: só um pacote "no ar" por vez, desperdiçando o enlace. **Solução: pipelining** — permitir múltiplos pacotes "em trânsito" simultaneamente, sem esperar confirmação individual de cada um, usando uma **janela** de tamanho J (o transmissor pode ter até J pacotes não confirmados ao mesmo tempo).

Go-Back-N (GBN)

- O **receptor** é simples: só aceita pacotes **em ordem estrita**. Qualquer pacote fora de ordem é **descartado** (mesmo que não esteja corrompido!), e o receptor reenvia um ACK **cumulativo** do último pacote recebido em ordem.
- **ACK cumulativo:** confirma "recebi tudo certinho até o número X" (não confirma pacotes individuais fora de ordem).
- Se o transmissor detecta perda (via timeout do pacote mais antigo não confirmado), ele **retransmite todos os pacotes da janela a partir daquele ponto** — daí o nome "volte N" (Go-Back-N): mesmo pacotes que já tinham chegado corretamente ao receptor (mas fora de ordem) são reenviados, porque o receptor os descartou.
- **Vantagem:** receptor simples (não precisa de buffer para reordenar). **Desvantagem:** desperdiça banda retransmitindo pacotes que talvez já tivessem chegado bem.

Números de sequência necessários: com janela de tamanho J, o espaço de números de sequência precisa ter pelo menos **J+1** valores diferentes (não pode ser igual a J), para o receptor conseguir diferenciar um pacote **novo** de uma **retransmissão antiga** que ainda esteja circulando na rede.

Selective Repeat (SR)

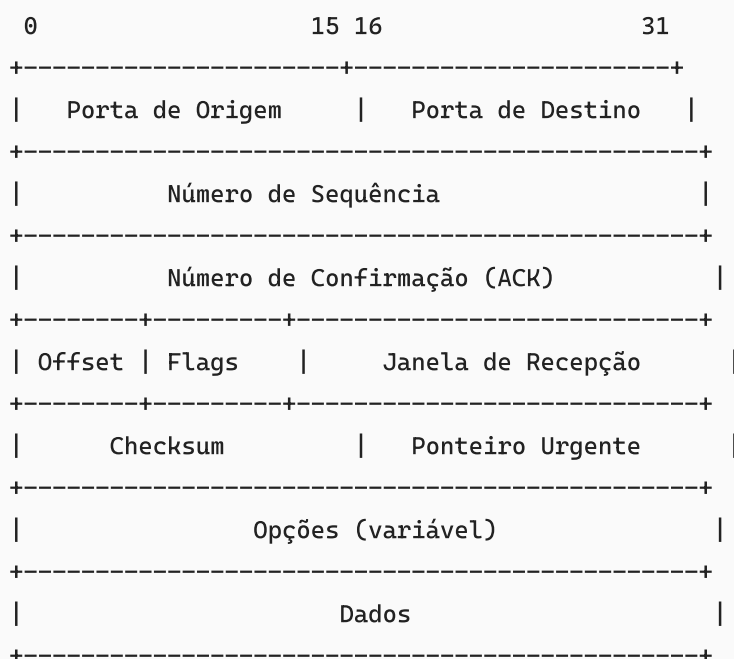
- O **receptor** é mais sofisticado: aceita e **armazena em buffer** pacotes fora de ordem (desde que estejam dentro da janela de recepção), enviando um **ACK individual** para cada pacote correto recebido — mesmo os fora de ordem.
- Quando o pacote faltante finalmente chega, o receptor consegue **reordenar** e entregar tudo em sequência para a camada de aplicação, sem precisar que o transmissor reenvie pacotes que **já** chegaram corretamente.
- **Retransmissão:** o transmissor só reenvia o(s) pacote(s) **especificamente** não confirmado(s) — daí "repetição seletiva".

- **Vantagem:** usa a banda de forma muito mais eficiente (menos retransmissões desnecessárias).
Desvantagem: receptor mais complexo (precisa de buffer de reordenação); e o espaço de números de sequência precisa ser **pelo menos o dobro do tamanho da janela (2J)**, porque tanto o transmissor quanto o receptor podem estar com visões parcialmente diferentes de quais pacotes já foram confirmados.

Conexão com o gabarito 2025/2 (Questão 1e): o "menor S" necessário para não confundir retransmissão com nova emissão depende de **o receptor descartar ou não pacotes fora de ordem**: se ele descarta (GBN), bastam **J** números de sequência adicionais ($S \geq J$); se ele aceita fora de ordem (SR, sem descartar), são necessários **2J**.

3.5 TCP — Estrutura e Funcionamento Básico

Formato do segmento TCP (cabeçalho de 20 bytes, sem opções)



Campos-chave que mais caem em prova:

- **Número de sequência:** identifica a posição, em **bytes**, do primeiro byte de dados daquele segmento **dentro do fluxo de bytes** da conexão (TCP não numera *pacotes*, numera *bytes*! Isso é diferente do RDT genérico que vimos acima, que numera pacotes).
- **Número de confirmação (ACK):** número do **próximo byte esperado** pelo receptor (ACK **cumulativo** — confirma tudo até ali, em ordem).
- **Janela de recepção (Receive Window):** usada para **controle de fluxo** (ver abaixo).
- **Flags:** incluem SYN (sincronizar/abrir conexão), ACK (confirmação válida), FIN (encerrar conexão), RST (resetar conexão abruptamente).

Gerenciamento de conexão: abertura (Three-Way Handshake)

Já vimos isso no contexto do HTTP, mas formalizando como conceito do TCP em si:

1. **Cliente → Servidor: segmento SYN** (com um número de sequência inicial escolhido aleatoriamente pelo cliente, `client_isn`)
2. **Servidor → Cliente: segmento SYN-ACK** (confirma o SYN do cliente com `ACK = client_isn+1` , e envia seu próprio número de sequência inicial `server_isn`)
3. **Cliente → Servidor: segmento ACK** (confirma o SYN do servidor com `ACK = server_isn+1`) — a partir daqui, dados já podem ser enviados junto com esse terceiro segmento

Por que 3 vias e não 2? Com apenas 2 vias (SYN, depois SYN-ACK), o **servidor** não teria confirmação de que o cliente realmente recebeu seu SYN-ACK — ele alocaria recursos (buffers, número de sequência inicial) para uma conexão que talvez o cliente nem tenha completado do outro lado. O terceiro segmento (ACK) garante que **ambos os lados** confirmaram a troca de números de sequência iniciais antes de qualquer dado real ser trocado.

Pergunta-armadilha clássica (aparece no gabarito 2022/1, Q4c): *"O three-way-handshake estabelece um circuito (circuit switch) half-duplex?"* — **FALSO**. O handshake estabelece uma conexão **lógica** (um estado compartilhado entre os dois lados, com números de sequência sincronizados), **não** uma reserva física de recursos como em comutação de circuitos — os pacotes TCP continuam trafegando por comutação de **pacotes**, disputando enlaces normalmente. Além disso, a conexão TCP é **full-duplex** (dados fluem nos dois sentidos simultaneamente), não half-duplex.

Gerenciamento de conexão: encerramento (Four-Way)

1. **Cliente → Servidor:** segmento FIN
2. **Servidor → Cliente:** ACK do FIN (o servidor pode ainda ter dados a enviar)
3. **Servidor → Cliente:** segmento FIN próprio, quando terminar de enviar tudo
4. **Cliente → Servidor:** ACK final

O encerramento precisa de **4** mensagens (não 3) porque a conexão é full-duplex — cada lado precisa encerrar seu **próprio** fluxo de envio de forma independente.

Controle de Fluxo (Flow Control) — não confundir com controle de congestionamento!

Ponto de atenção crítico (pegadinha recorrente — gabarito 2022/1, Q4b): controle de fluxo e controle de congestionamento **não são a mesma coisa**, e a afirmação "controle de congestionamento evita gargalo no receptor, controle de fluxo evita gargalo no emissor" está **invertida e errada**.

- **Controle de FLUXO:** evita que o **transmissor sobrecarregue o BUFFER do RECEPTOR** — se o receptor for lento para processar os dados (aplicação lenta lendo do socket), seu buffer pode encher. O receptor anuncia, no campo **janela de recepção** do cabeçalho TCP, **quanto espaço livre ainda tem** no seu buffer — e o transmissor nunca envia mais dados do que esse valor permite.

- **Controle de CONGESTIONAMENTO** (fora do escopo desta prova, mas para não confundir): evita que o transmissor sobrecarregue a **rede** (roteadores/enlaces no meio do caminho), não o receptor.

Resumindo a tabela de confusão: controle de fluxo = problema é o **receptor** (fim a fim, dentro do TCP); controle de congestionamento = problema é a **rede** (os enlaces/roteadores no meio do caminho). Nenhum dos dois tem relação com comutação de circuitos ou de pacotes como "abordagem" — ambos operam dentro de uma rede de comutação de **pacotes**.

Questões de Provas Anteriores — Resolução Detalhada Passo a Passo

Selecionadas das provas de 2022/1, 2022/2, 2024/2, 2025/1, 2025/2 e 2026/1 por recorrência temática. Cada resolução mostra **todo** o raciocínio, sem pular etapas — o objetivo é que alguém sem nenhum conhecimento prévio consiga acompanhar.

Questão A — Dedução completa da Lei de Little + fila M/M/1 (tema mais recorrente de todas as provas)

(Baseado em 2025/1 Q1 e no "preencha as lacunas" de 2025/2 Q3)

Enunciado resumido: Explique a Lei de Little e sua aplicação em filas de roteadores. Deduza . Calcule para bits, pacotes/s, Mbps.

Passo 1 — O que é a Lei de Little

A Lei de Little é uma relação de **conservação**, válida para **qualquer** sistema de fila em equilíbrio, independente da distribuição de chegadas ou de serviço:

onde L é o número médio de itens no sistema, λ é a taxa média de chegada, e W é o tempo médio que cada item passa no sistema. **Intuição:** numa janela de tempo longa T , chegam aproximadamente λT pacotes; se cada um passa W segundos no sistema, o total de "pacote-segundos" acumulados é $\lambda T W$. Por outro lado, se em média há L pacotes no sistema a cada instante, a mesma área acumulada é $L T$. Igualando: $L T = \lambda T W$.

Passo 2 — Por que ela se aplica a filas de roteadores

Um roteador com um enlace de saída é modelado como um sistema de fila com 1 servidor: pacotes chegam a uma taxa λ , esperam se o enlace estiver ocupado, e são "servidos" (transmitidos) a uma taxa μ . A Lei de Little se aplica diretamente a esse sistema, tanto ao sistema completo (fila+servidor) quanto só à fila.

Passo 3 — Deduzindo (modelo M/M/1)

Um pacote que chega encontra, em média, pacotes na fila (cada um exigindo um tempo de serviço completo até ele) e o servidor ocupado com probabilidade (contribuindo, por falta de memória da exponencial, com um tempo residual médio igual ao tempo de serviço completo). Logo, o tempo médio de espera na fila:

Pela Lei de Little aplicada só à fila: . Substituindo:

Como :

O tempo total no sistema:

Pela Lei de Little ao sistema completo: . Como :

□

Passo 4 — Número médio no buffer (só fila, sem o pacote em serviço)

Passo 5 — Aplicando os números

bits, pacotes/s, bits/s.

□
é

□
é o

Questão B — Cache antes do roteador, com redução de capacidade (Q1 completa, prova 2026/1)

Roteador modelado como fila; ; ; (Little).

(a) Cache reduz a chegada pela metade: . Encontre em função de , e mostre .

A nova intensidade de tráfego:

Expressando em função de N: de , isolamos . Substituindo:

Mostrando que : como , temos , logo **sempre**, para qualquer original. ■

Interpretação intuitiva: não importa o quão congestionado o sistema **original** estivesse (mesmo com grande, sistema quase saturado), depois de **cortar a taxa de chegada pela metade**, o número médio de pacotes no roteador **nunca** ultrapassa 1 — o cache "tira" o sistema da zona de explosão não-linear de perto de .

(b) Cache reduz E capacidade reduz . Encontre e .

Para : como (tempo de serviço **dobra**, já que a capacidade caiu pela metade):

A cache compensou a redução de capacidade? significa que o número médio de pacotes **não mudou** — parece que sim, compensou perfeitamente. Mas mostra que o **tempo** que cada pacote passa no sistema **dobrou**! A cache manteve a "ocupação" do roteador igual, mas cada pacote individual demora o dobro para ser atendido, porque o enlace ficou mais lento. **Não houve compensação real do ponto de vista do usuário** — a percepção de atraso piorou, mesmo com N igual.

(c) Números: Mbit, pac/s, Mbit/s

(i) Sistema original:

(ii) Cache ():

(iii) Banda dobrada ():

(iv) Cache + banda reduzida (,):

(v) **Comparação qualitativa (assumindo hit da cache tem custo desprezível):** o cenário (iii) dobrar a banda é o **melhor** para o usuário (menor $T=6,67\text{ms}$, sem mexer na taxa de chegada real de misses). O cenário (ii), cache sozinha, também melhora bem (T caiu de 20 para $13,3\text{ms}$) — e como metade das requisições nem chega a passar pelo roteador (acerto de cache, custo desprezível), o **atraso médio percebido pelo usuário** é ainda menor que os $13,3\text{ms}$ calculados (que só valem para os misses). O cenário (iv) é o **pior**: mesmo com a cache ajudando, a redução de capacidade anula o ganho e ainda dobra o T em relação ao original.

(d) DNS recursivo vs. iterativo

Já cobrimos essa explicação completa na seção 2.4.3 acima — **iterativo favorece o preenchimento do cache do servidor local**, porque ele vê e armazena cada resposta intermediária da hierarquia, diferente do recursivo, que só recebe a resposta final pronta.

(e) Espaço médio ocupado pela cache DNS (TTL)

Requisições chegam a taxa λ ; hit com probabilidade h ; miss com probabilidade $1-h$ cria uma nova entrada; cada entrada fica em média $\frac{1}{\mu}$ segundos e ocupa α bits.

Passo 1: a taxa de **criação** de novas entradas na cache é a taxa de misses: $\lambda(1-h)$.

Passo 2: cada entrada, uma vez criada, permanece por um tempo médio antes de expirar (TTL). Isso é **exatamente** a estrutura da Lei de Little: "entradas" são como "pacotes no sistema", a "taxa de chegada" é $\lambda(1-h)$, e o "tempo no sistema" é $\frac{1}{\mu}$. Logo, o **número médio de entradas** na cache em um dado instante:

Passo 3: cada entrada ocupa α bits, então o **espaço médio ocupado**:

çé

Questão C — Verdadeiro ou Falso: as pegadinhas mais recorrentes (compiladas de 2026/1, 2025/1, 2024/2, 2022/1)

1. "O TCP é stateless, pois roda sobre uma rede packet switching que não mantém estado." → **FALSO**. TCP é **stateful** — mantém números de sequência, janela, estado da conexão (estabelecida, fechando, etc.) durante toda a vida da conexão. O tipo de comutação da rede subjacente (packet switching) não determina se o protocolo de transporte por cima é stateful ou não — são camadas diferentes, com decisões de design independentes.
2. "O UDP é stateful porque usa checksum." → **FALSO**. Checksum verifica a integridade de **um único datagrama isolado** — não implica guardar nenhuma informação entre datagramas diferentes. UDP continua sem conexão e sem estado.
3. "HTTP é stateful, motivando o uso de cookies." → **FALSO (invertido)**. HTTP é **stateless** — e é **por isso** que cookies existem: para **compensar** a ausência nativa de estado.
4. "O núcleo da Internet é baseado em protocolos stateful para garantir confiabilidade." → **FALSO**. O núcleo (roteadores, protocolo IP) é **stateless por design** — cada roteador trata cada datagrama IP de forma independente ("best effort", sem lembrar de pacotes anteriores). A confiabilidade, quando existe, é garantida **fim a fim** (pelo TCP nos hosts extremos), não dentro do núcleo.
5. "Controle de congestionamento evita gargalo no receptor; controle de fluxo evita gargalo no emissor; congestionamento usa circuit switching, fluxo usa packet switching." → **FALSO em tudo**. Invertido: controle de **fluxo** protege o **receptor** (buffer dele); controle de **congestionamento** protege a **rede** (enlaces/roteadores no meio). Nenhum dos dois usa "abordagem circuit switching" — ambos operam dentro de uma rede de comutação de pacotes, apenas ajustando a taxa/janela de envio.
6. "Three-way handshake estabelece um circuito half-duplex." → **FALSO**. Estabelece uma conexão **lógica** (não uma reserva física de recursos como em comutação de circuitos), e a conexão resultante é **full-duplex** (dados nos dois sentidos simultaneamente), não half-duplex.
7. "HFC usa FDMA, sendo portanto circuit switching." → **FALSO**. FDMA é uma **técnica de multiplexação** (divide o espectro em faixas), mas isso **não implica automaticamente** comutação de circuitos — o cabo HFC é compartilhado entre vizinhos de forma que pode envolver disputa/contenção (mais próximo do espírito de packet switching no acesso), mesmo usando FDM para separar canais.
8. "HTTP/2.0 é stateful porque permite priorização de requisições, enquanto HTTP/1.0 é stateless." → **FALSO**. Priorização de requisições é um recurso de **como a conexão TCP é usada** (interleaving), não muda a semântica da camada de aplicação HTTP, que continua **stateless em ambas as versões**.
9. "Basta enviar o cookie na primeira mensagem TCP para ele funcionar." → **FALSO**. O cookie precisa ser enviado pelo **servidor** na resposta HTTP (Set-Cookie:), e devolvido pelo **cliente** em **toda requisição HTTP subsequente** (Cookie:) — não é algo enviado "uma vez só" na conexão TCP; é repetido a cada requisição HTTP, porque HTTP é stateless e cada requisição precisa "reapresentar" o cookie.

10. "O comando close gera tráfego ao fechar sockets TCP e UDP." → **PARCIALMENTE FALSO** — **pegadinha**. Fechar um socket **TCP** gera tráfego real (segmentos FIN/ACK do encerramento de 4 vias). Fechar um socket **UDP**, por outro lado, é uma operação **puramente local** (não há conexão estabelecida para "encerrar" com o outro lado) — **não gera nenhum tráfego de rede**.

Questão D — Probabilidade de sucesso e retransmissões com perda por enlace (2022/1, itens d-e)

N enlaces em série, cada um com probabilidade de perda independente.

(d) Probabilidade de um pacote chegar com sucesso ao cliente:

Para ter sucesso, o pacote precisa **sobreviver a todos os N enlaces**. Como as perdas são independentes, multiplicamos as probabilidades de sucesso individual em cada enlace:

(e) Probabilidade de exatamente k retransmissões até o sucesso:

Isso é uma **Distribuição Geométrica**: "k falhas seguidas, depois um sucesso". Seja a probabilidade de sucesso em **uma** tentativa (uma travessia completa dos N enlaces). A probabilidade de exatamente retransmissões (ou seja, falhas antes do sucesso, totalizando tentativas) é:

Atenção à contagem: significa "acertou de primeira, sem nenhuma retransmissão" — por isso o suporte começa em aqui (não em 1 como na Geométrica "padrão" que conta *tentativas*, já que aqui a variável conta *retransmissões*, que é tentativas-1).

Questão E — Checksum de 2 bits detectando erro de 1 bit (2022/1, Q3a)

Mensagem original de 2 bits. Quantos bits de checksum detectam corrupção de até 1 bit?

Passo 1 — Ideia do checksum de paridade: para detectar **qualquer** erro de 1 bit, um checksum de **1 bit de paridade** já basta: soma-se (XOR) os bits da mensagem, e esse resultado é o bit de paridade.

Exemplo concreto: mensagem de 2 bits , bit de paridade .

MENSAGEM ORIGINAL ()	PARIDADE	TRANSMITIDO (3 BITS)
00	0	000
01	1	011

MENSAGEM ORIGINAL ()	PARIDADE	TRANSMITIDO (3 BITS)
10	1	101
11	0	110

Se **um único bit** (qualquer um dos 3) for invertido por corrupção, o XOR de todos os 3 bits recebidos deixa de bater com 0 (paridade par esperada) — detectando o erro.

O que acontece se o bit de overhead (paridade) for corrompido, e não os dados? A detecção **ainda funciona**: a verificação de paridade no receptor vai acusar inconsistência de qualquer forma, mesmo que o erro esteja no próprio bit de paridade — porque a verificação olha para a **consistência entre todos os 3 bits recebidos**, não sabe (nem precisa saber) qual bit especificamente foi corrompido.

Overhead adicional necessário além do checksum: sim — checksum sozinho só **detecta** erro, não é suficiente para uma transferência confiável completa. É necessário também: **números de sequência** (para diferenciar pacotes novos de retransmissões) e o mecanismo de **ACK/NAK + timeout** (para o transmissor saber que precisa retransmitir).

Questão F — Blocos vs. Fluido no P2P (2022/2) — já resolvida na parte teórica

Ver seção 2.5 acima, "Interpretação em Termos de Redes (Análise com Uploads Iguais)" — mostra que no limite com uploads iguais, (não cresce com N), demonstrando formalmente a autoescalabilidade.

Questão G — GBN: números de sequência possíveis na janela (2025/2, Q1a — dedução completa)

, (sequências possíveis: 0, 1, 2). Receptor espera pacote 0. Quais sequências podem estar na janela do emissor?

Passo 1: com , o emissor só pode ter **1 pacote pendente de ACK por vez** — comporta-se, na prática, como stop-and-wait.

Passo 2: se o receptor está esperando o pacote **0**, existem exatamente **dois cenários possíveis** para o que o emissor tem na janela:

- **Cenário 1 (início de novo ciclo):** o emissor já recebeu o ACK do pacote 2 (o anterior ao 0 no ciclo, em aritmética módulo 3) e **já enviou** o novo pacote 0 — nesse caso, a janela do emissor contém o pacote **0**.
- **Cenário 2 (ACK ainda não chegou):** o receptor já recebeu com sucesso os pacotes 0, 1 e 2 (por isso está esperando o próximo, que seria o 0 do ciclo seguinte), mas o **ACK do pacote 2** ainda está

a caminho de volta ao emissor — nesse caso, o emissor **ainda está esperando** a confirmação do pacote **2**, que continua sendo o último pacote na sua janela.

Por que o pacote 1 NÃO pode estar na janela: se o emissor tivesse o pacote 1 pendente, isso significaria que o **último pacote confirmado** pelo emissor foi o 0 — e nesse caso, o receptor já teria recebido o 0 e estaria esperando o **1**, não o 0. Contradição com o enunciado (receptor espera 0).

Resposta: a janela do emissor, no instante t , contém ou o pacote 0, ou o pacote 2 (nunca o 1).